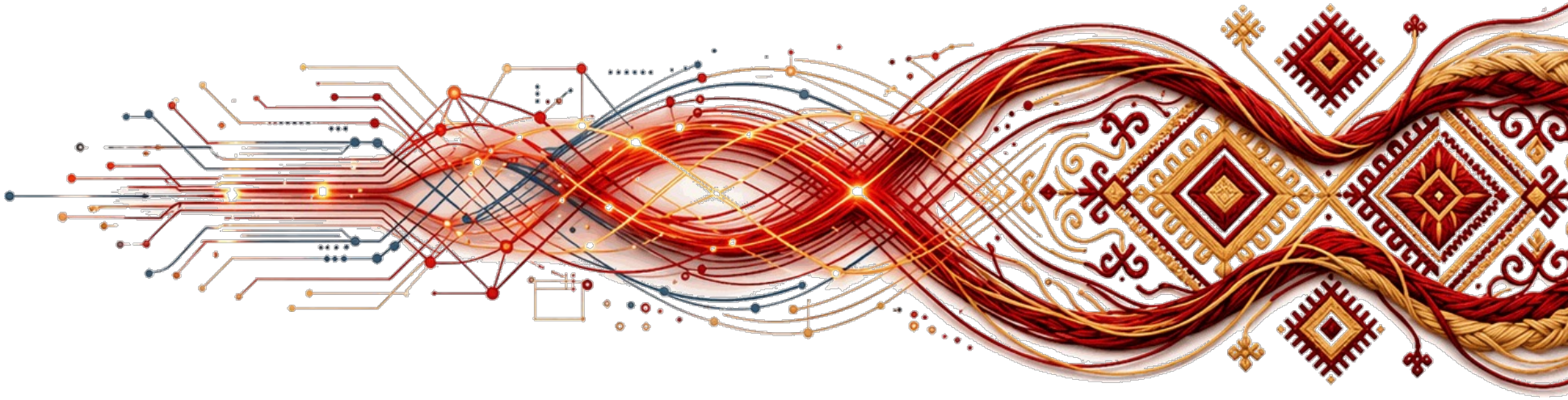
**Introduction to Data Science**

BASIC TRACK · LECTURE 4



# Supervised learning I: regression

## Predicting a numerical response: models, estimation and regularisation

**Prof. Dimitar Trajanov, PhD**
**Prof. Igor Mishkovski, PhD**
**Prof. Andrea Kulakov, PhD**
**Prof. Slobodan Kalajdziski, PhD**

"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Vezilka · AI Factories · EuroHPC Joint Undertaking · Co-funded by the European Union · Republic of North Macedonia Ministry of Digital Transformation

# Agenda

1. **Learning from data**
   What regression is; response, predictors, model
2. **The k-nearest-neighbour method**
   Predicting from the most similar examples
3. **Error evaluation and model fit**
   Train–test split, MSE, RMSE, MAE, $R^2$
4. **Linear regression**
   Least squares by hand, closed form, gradient descent
5. **Polynomial regression and overfitting**
   Underfitting, overfitting, bias and variance
6. **Regularisation**
   Ridge, lasso, elastic net; λ by cross-validation
7. **A regression analysis in practice**
   The workflow, test error and the coefficients

# Learning objectives of the lecture

- After this lecture, the student is able to:
  - Describe a learning algorithm by its **representation, evaluation and optimisation**.
  - Distinguish the response from the predictors, and **prediction from estimation**.
  - Fit a **k-nearest-neighbour** regressor and choose k on a validation set.
  - Compute **MSE, RMSE and $R^2$** and interpret them against a baseline.
  - Estimate a linear regression by the **closed form** and by **gradient descent**.
  - Explain overfitting and control it with **ridge, lasso and elastic net**, choosing λ by cross-validation.
  - Interpret the coefficients of a multiple regression.

# 01

# Learning from data and the regression problem

The components of a learning algorithm and the formulation of a numerical prediction problem



# Regression in everyday words: predicting a number

**1 House price**

Input: area, location, age of the flat.
Output: the price in EUR.

**2 Sales**

Input: TV, radio and newspaper budgets.
Output: units sold (this lecture).

**3 Exam score**

Input: hours studied, attendance.
Output: points out of 100.

**4 Electricity**

Input: temperature, day of the week.
Output: kWh used tomorrow.




- *Regression*: the answer is a **number**. *Classification* (Lecture 5): the answer is a **category**, such as spam / not spam.
- Both are *supervised learning*: the model learns from examples whose correct answer is known, then answers for **new** cases.

# Key words and symbols used in this lecture

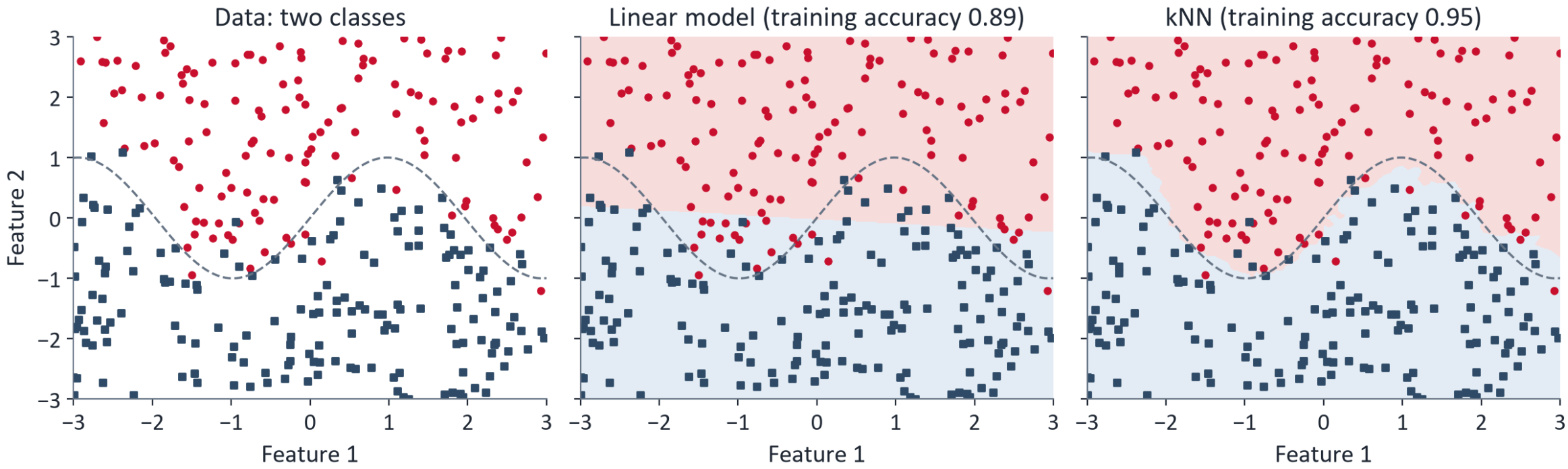
| Term | Meaning | Example: Advertising data |
|---|---|---|
| Observation (row) | One case in the data | One market |
| Response y | The number we want to predict | Sales, in thousand units |
| Predictor x | A variable used to predict y; also called a *feature* | TV budget, in thousand $ |
| Model $\hat{f}$ | The rule learned from the data | $\hat{y} = 7.03 + 0.0475 \cdot TV$ |
| Prediction $\hat{y}$ ("y-hat") | The model's answer for one input | TV = 100 → $\hat{y} = 11.79$ |
| Residual $e = y - \hat{y}$ | Actual minus predicted: the error on one row | $y = 13.00$, $\hat{y} = 11.79$ → $e = 1.21$ |
| Coefficient β | A number inside the model, learned from the data | slope $\beta_1 = 0.0475$ |
| Hyperparameter | A setting chosen by us *before* fitting | k in kNN, λ in ridge and lasso |
| Training / test data | Rows used to learn / rows kept aside to check | 150 / 50 markets |

# The three components of a learning algorithm

| Representation | + | Evaluation | + | Optimisation |
|---|---|---|---|---|
| *The family of models searched* | | *The score that ranks models* | | *The search for the best model* |
| • Linear models<br>• Instances (k-nearest neighbours)<br>• Decision trees and rules<br>• Neural networks<br>• Model ensembles | | • Squared error<br>• Accuracy, precision, recall<br>• Likelihood<br>• Margin, entropy<br>• Cost or utility | | • Exhaustive or grid search<br>• Closed-form solution<br>• Gradient descent<br>• Greedy search<br>• Linear programming |
| **In this lecture: Linear model $\hat{y} = \beta_0 + \beta_1 x$** | | **In this lecture: Mean squared error** | | **In this lecture: Closed form, gradient descent** |

- *Supervised learning* uses labelled examples to learn **a predictor**; its components are representation, evaluation and optimisation.
  - The columns distinguish the model family, the objective and the search method; the red boxes identify the regression examples.

# Linear versus non-linear models




- A linear model separates the classes with **a straight boundary** (training accuracy 0.89); kNN has a flexible boundary (training accuracy 0.95).
  - The simulated classification example illustrates geometry only; these training scores do not establish predictive superiority on new data.

# Response and predictors in the Advertising data

- The *response* Y is the variable to be predicted; the *predictors* $X_1$, …, X_p are **the variables used to predict it**.
  - Synonyms: outcome or dependent variable; features or covariates.
- The Advertising data record **sales in n = 200 markets** with p = 3 predictors.
  - Budgets for TV, radio and newspaper, in thousands of dollars; sales in thousands of units.
- The data form an n × p matrix X and a response vector y of length n.
  - Mean sales are **14.02**; TV shows the strongest association.
- Each row is one market, so the rows are **the units of analysis**.

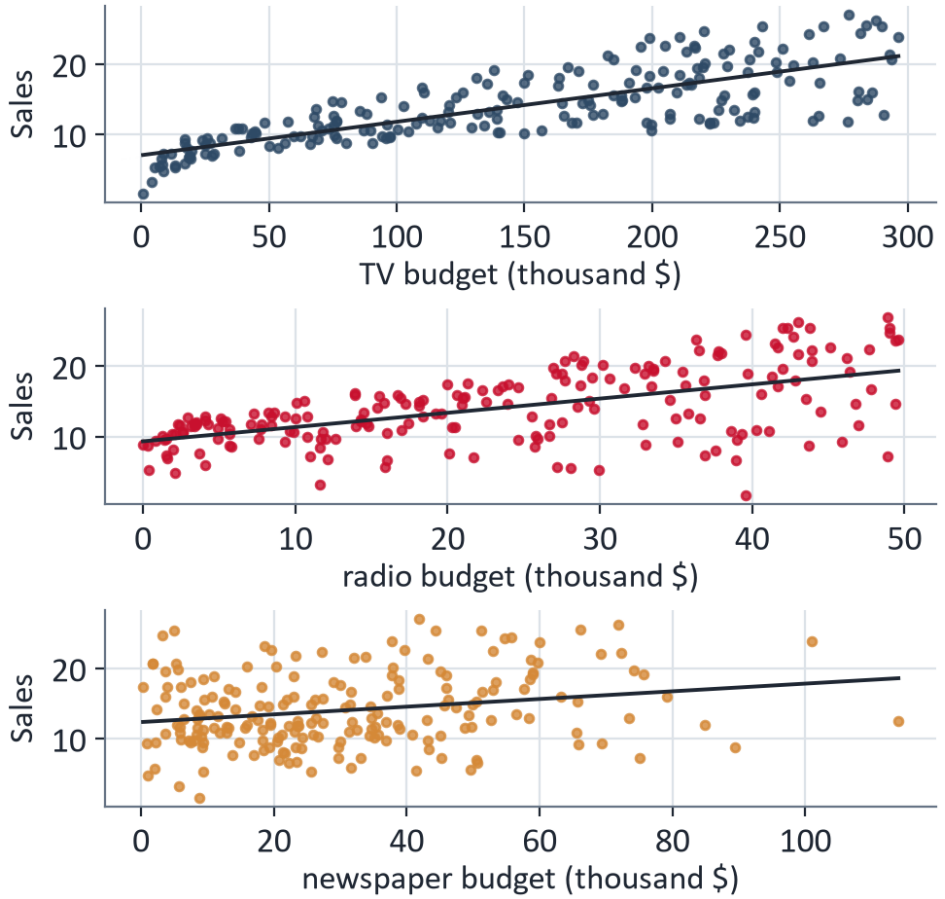



*Figure 1 — sales against each advertising budget*

# The true model and the statistical model

- The *true model* $Y = f(X) + \varepsilon$ links the response to the predictors through **an unknown function f**.
  - For squared-error prediction, $f(X) = E[Y \mid X]$ and $E[\varepsilon \mid X] = 0$; the error records deviations from this conditional mean.
- A *statistical model* $\hat{f}$ is an estimate of f computed from the sample.
  - The figure adds three estimates on successive clicks: the mean, a straight line and **a k-nearest-neighbour curve**.
- The task of *regression* is to estimate f when **Y is numerical**; a categorical Y defines classification (Lecture 5).
- Choosing among candidate models requires **a measure of error**, defined in Section 03.
  - The three candidates differ in flexibility, not in the data they use.

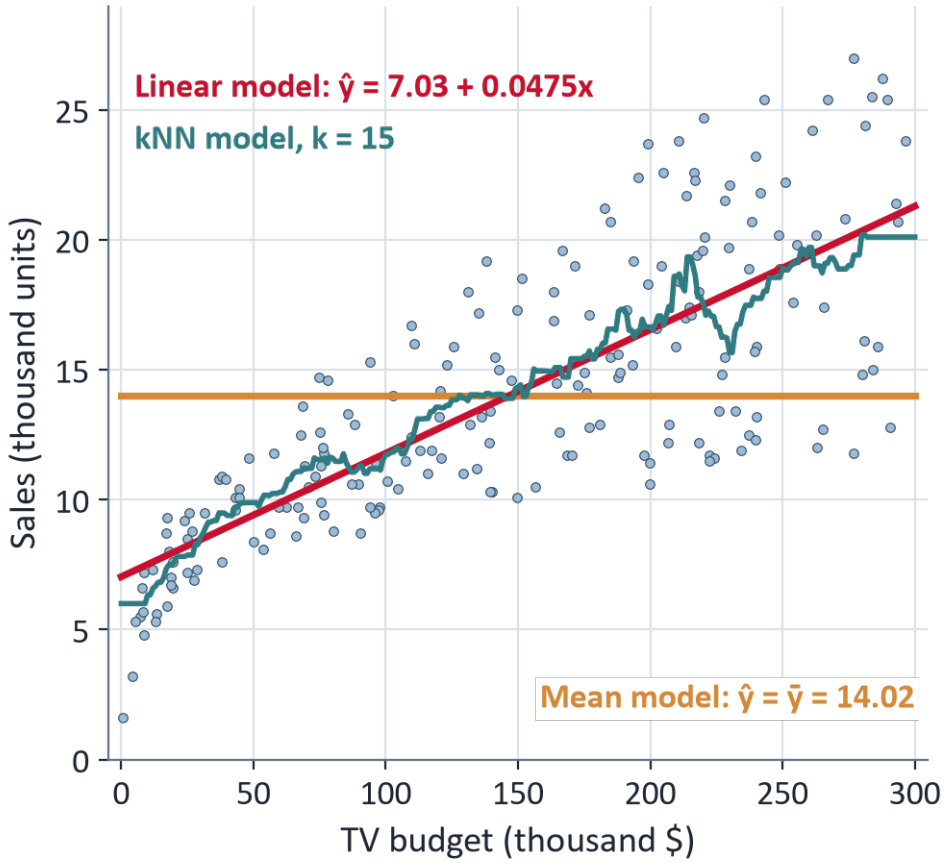



*Figure 2 — three estimates of f on one sample*

# Prediction versus estimation

## Prediction

- The goal is **an accurate value $\hat{y} = \hat{f}(x)$** for a new input x.
  - Example: the sales expected in a market with a given TV budget.
- Predictive accuracy does not by itself require an interpretable model.
  - Flexible models such as kNN or boosted trees are acceptable here.
- Success is measured by **the error on unseen data**.
- Evaluation must match deployment conditions, including time, market coverage and the **cost of errors**.

## Estimation (inference)

- The goal is **to estimate model parameters** and quantify their uncertainty.
  - Example: the change in sales associated with one more thousand dollars of radio advertising.
- A linear coefficient describes a conditional association under the specified model; it does not automatically identify a causal effect.
  - Section 07 interprets the Advertising coefficients.
- Inference requires **valid assumptions and calibrated uncertainty**, not merely small standard errors.

02

# The k-nearest-neighbour method

Nearest-neighbour prediction uses similar training observations for classification and regression.



# kNN in plain words: ask the most similar neighbours

**1 New case**

A flat of 62 m² is for sale. What is a fair price?

**2 Distance**

Compare its size with every flat sold recently.

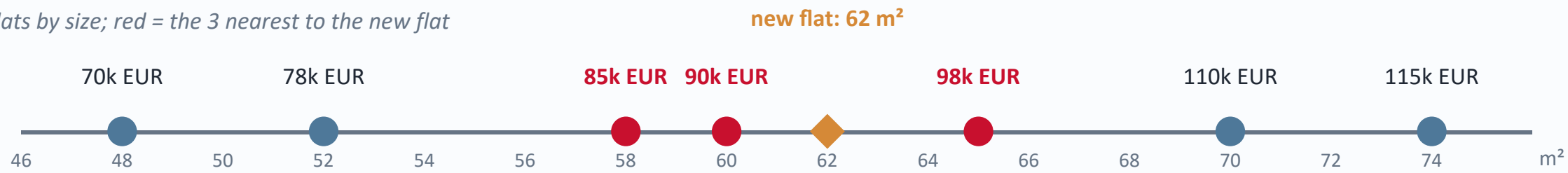
**3 k nearest**

With k = 3: the sold flats of 60, 65 and 58 m² (2, 3 and 4 m² away).

**4 Average**

Their prices 90, 98 and 85 thousand EUR:
$\hat{y} = (90 + 98 + 85) / 3 = 91$ thousand EUR.

*Sold flats by size; red = the 3 nearest to the new flat*




- kNN learns **no equation**: it remembers the training rows and decides when a question arrives.
- **Small k** follows single neighbours (noisy); **large k** averages many (smooth); k = all rows gives the plain mean.
- Predictors must be on **comparable scales**: 1 m² and 1 EUR cannot be compared directly, so they are standardised.

# The k-nearest-neighbour algorithm

- The *k-nearest-neighbour* (kNN) method predicts from the **k training rows closest** to the query.
  - A *query* is the new predictor vector whose response is required.
  - Compute the distance from the query to every training row, sort, and keep the k smallest.
  - Classification: majority vote; regression: **mean of the k responses**.
- Distance is usually Euclidean:
  $d(a, b) = \sqrt{[(a_1 - b_1)^2 + (a_2 - b_2)^2 + \ldots]}$.
  - The unknown film lies 18.9, 19.2 and 20.5 from three romances; with k = 3 it is **classified as a romance**.
- This Euclidean implementation requires numeric features, handling of missing values and **comparable scales**.
  - Without scaling, a feature with **a large numerical range** can dominate the distance; scaling statistics must be fitted on training rows.

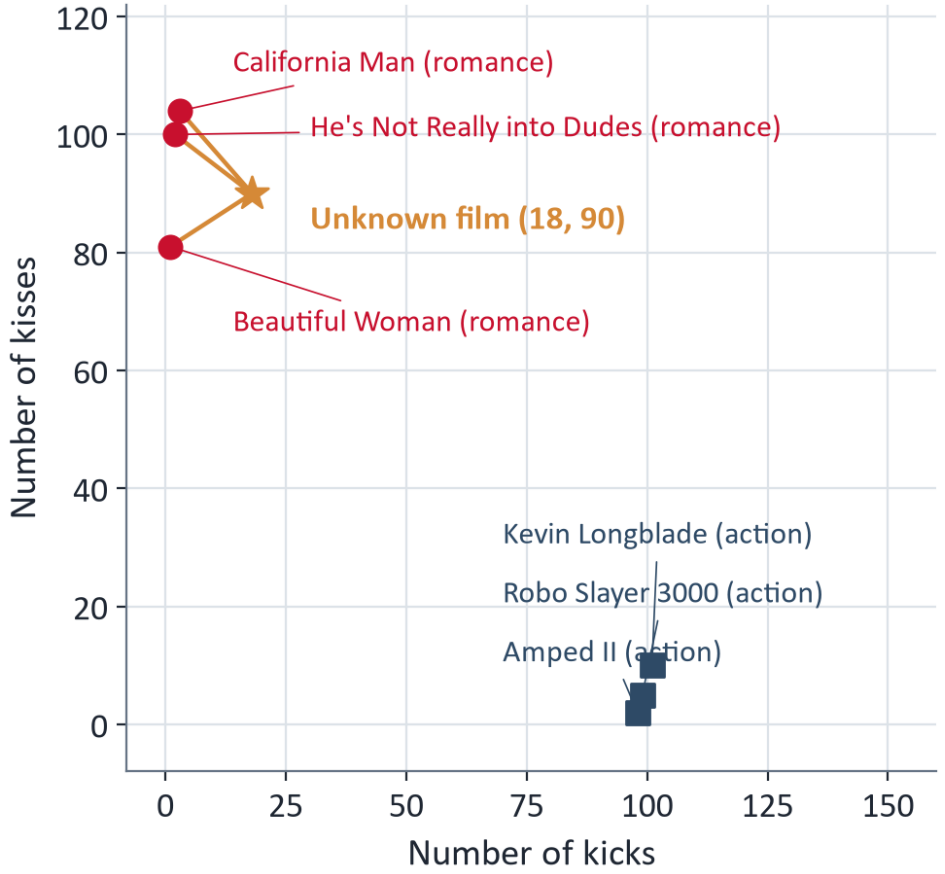



*Figure 3 — film genres*

# Nearest-neighbour regression for one query point

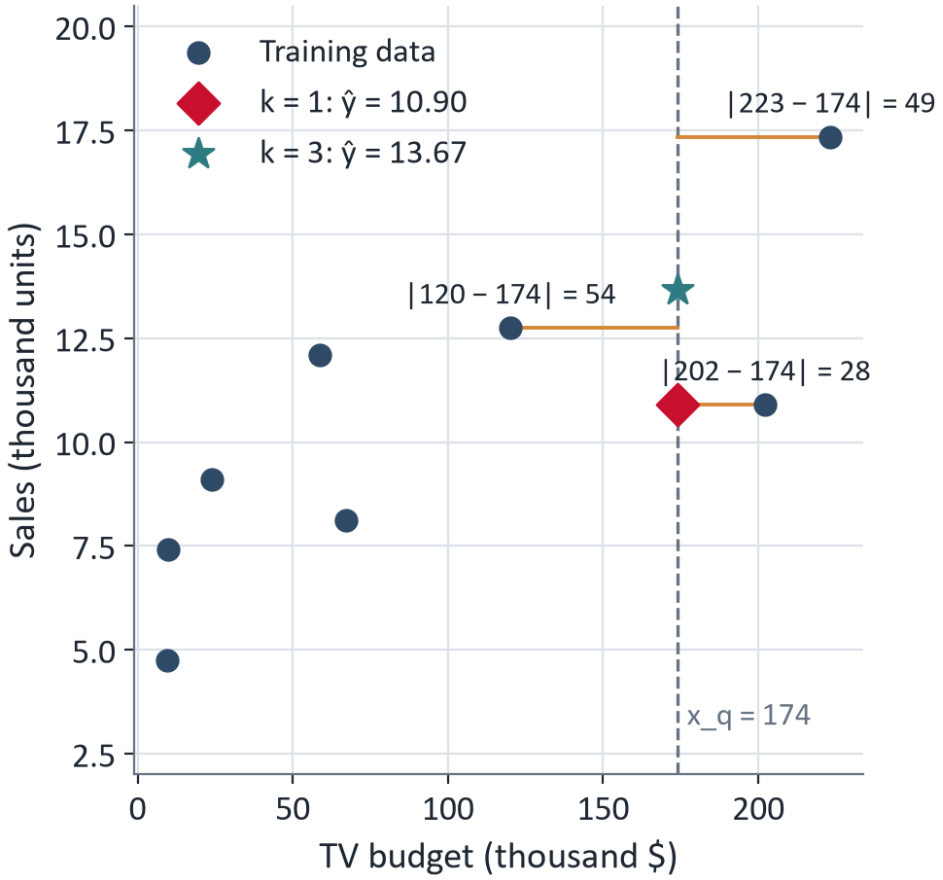
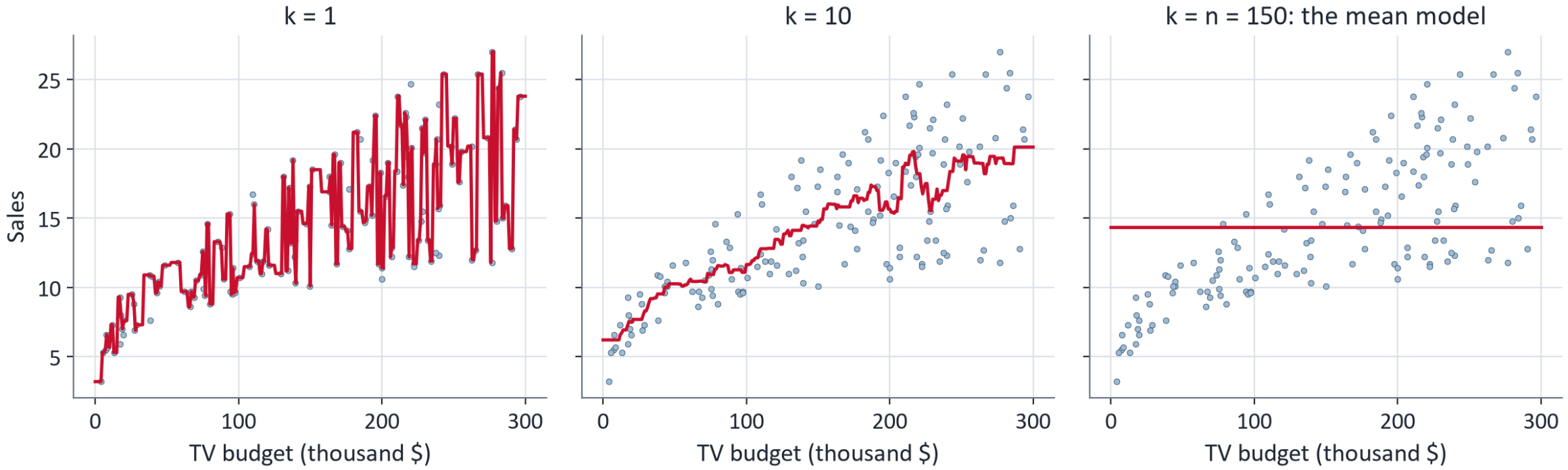



*Figure 4 — eight markets, query x_q = 174*

$$\hat{y}(x_q) = \underbrace{\frac{1}{k}\sum_{i \in N_k(x_q)}}_{\text{average over the k nearest points}} \underbrace{y_i}_{\text{their responses}}$$

- The prediction for a query x_q is the **average response of its k nearest** training points.
  - With k = 1 the nearest budget is 202, at distance 28, so **ŷ = 10.90**.
  - With k = 3 the neighbours are 202, 223 and 120, so ŷ = (10.9 + 17.35 + 12.75) / 3 = **13.67**.
- The method stores training data rather than estimating **a global coefficient vector**.
  - A brute-force query computes one distance per row; tree indexes can accelerate queries in suitable dimensions.
- The *query point* x_q is the input for which a prediction is required.

# The number of neighbours and the shape of the fit




- With distinct inputs, k = 1 interpolates training responses; larger k averages wider neighbourhoods and, with k = n, **reduces to the mean model**.
  - A *hyperparameter* controls the fitting procedure rather than being a fitted coefficient; k is selected using validation data (Section 03).

# 03

# Error evaluation and model fit

The models of Sections 01 and 02 are compared using error measures on held-out observations.



# Why we test on unseen data: the exam analogy

## Memorising the answers

- A student learns the practice questions **by heart**.
- Score on the practice questions: **100 %**.
- Score on the real exam, with new questions: **poor**.
- A model that does this is like **kNN with k = 1**: zero training error, large test error.
- *Training error* = the practice score: it flatters the model.

## Understanding the material

- A student learns the **general rules** behind the questions.
- Score on the practice questions: good, not perfect.
- Score on the real exam: **also good**, because the rules carry over.
- This is what we want from a model: a **small error on new data**.
- *Test error* = the exam score: the honest measure of quality.

# The train–test split for error evaluation

- A *train–test split* hides part of the data from the model and measures the error **only on the hidden part**.
  - The split size depends on sample size and the precision required for evaluation; grouped or time-dependent data require matching splits.
- The kNN fit with k = 1 passes through every training point, so its **training error is zero**.
  - A zero training error says nothing about future markets.
- The red dotted segments are the test errors; their mean square is **11.65**.
  - 5 of the 18 observations were drawn at random for the test set.
- The test part must **never influence fitting or tuning**, or its error is optimistic again.

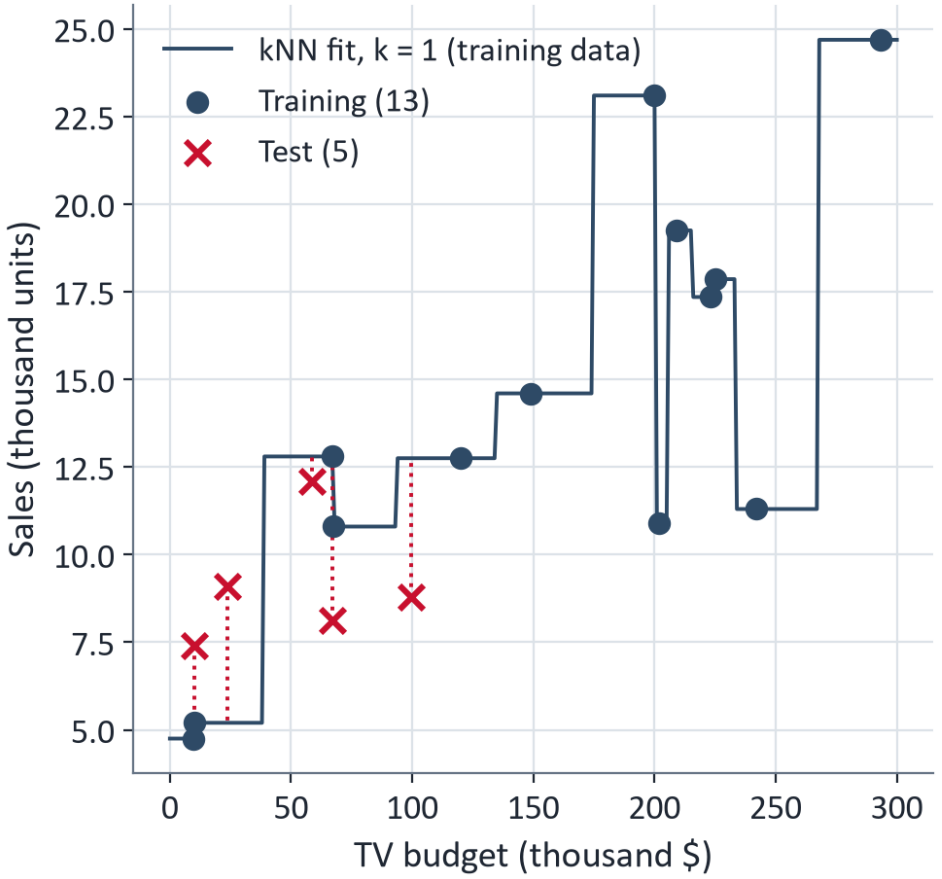



*Figure 5 — 18 observations, 13 train and 5 test*

# Mean squared error and root mean squared error

For n observed values $y_i$ and their predictions $\hat{y}_i$:

$$\mathrm{MSE} = \underbrace{\frac{1}{n}\sum_{i=1}^{n}}_{\text{average over the n rows}} \underbrace{(y_i - \hat{y}_i)^2}_{\text{squared error of row i}}$$

$$\mathrm{RMSE} = \underbrace{\sqrt{\mathrm{MSE}}}_{\text{back in the units of y}}$$

- Squared errors are nonnegative and **penalise large residual magnitudes quadratically**.
  - The *mean absolute error* $(1/n)\ \Sigma\ |y_i - \hat{y}_i|$ is less sensitive to outliers.
  - RMSE is never smaller than the mean absolute error; **a large gap** indicates uneven absolute error sizes, not necessarily a few outliers.

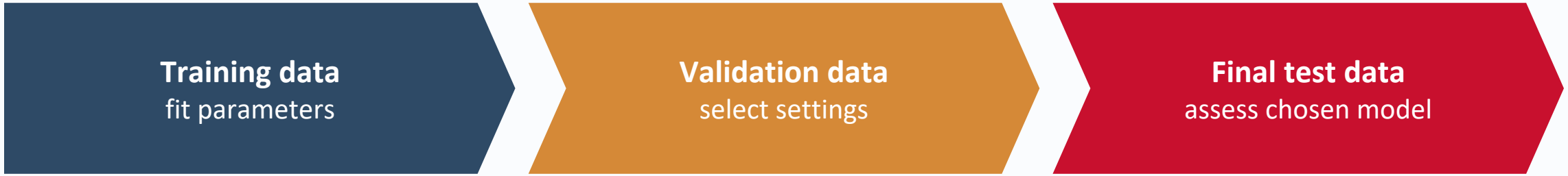
# Worked example: MSE, RMSE and MAE step by step

| x | y | $\hat{y} = 2.2 + 0.6x$ | $e = y - \hat{y}$ | $e^2$ | $\lvert e \rvert$ |
|---|---|---|---|---|---|
| 1 | 2 | 2.80 | −0.80 | 0.64 | 0.80 |
| 2 | 4 | 3.40 | 0.60 | 0.36 | 0.60 |
| 3 | 5 | 4.00 | 1.00 | 1.00 | 1.00 |
| 4 | 4 | 4.60 | −0.60 | 0.36 | 0.60 |
| 5 | 5 | 5.20 | −0.20 | 0.04 | 0.20 |
| | | | **Sum** | **2.40** | **3.20** |

1. **Errors**
   $e = y - \hat{y}$ for every point
2. **Square and add**
   $\Sigma e^2 = 2.40$
3. **Average → MSE**
   MSE = 2.40 / 5 = 0.48
4. **Square root → RMSE**
   RMSE = √0.48 = 0.69
5. **Absolute values → MAE**
   MAE = 3.20 / 5 = 0.64

- Read RMSE as **the typical size of an error**, in the units of y: these predictions miss by about 0.7.
- Squaring lets one big miss count more than several small ones, so RMSE (0.69) ≥ MAE (0.64).

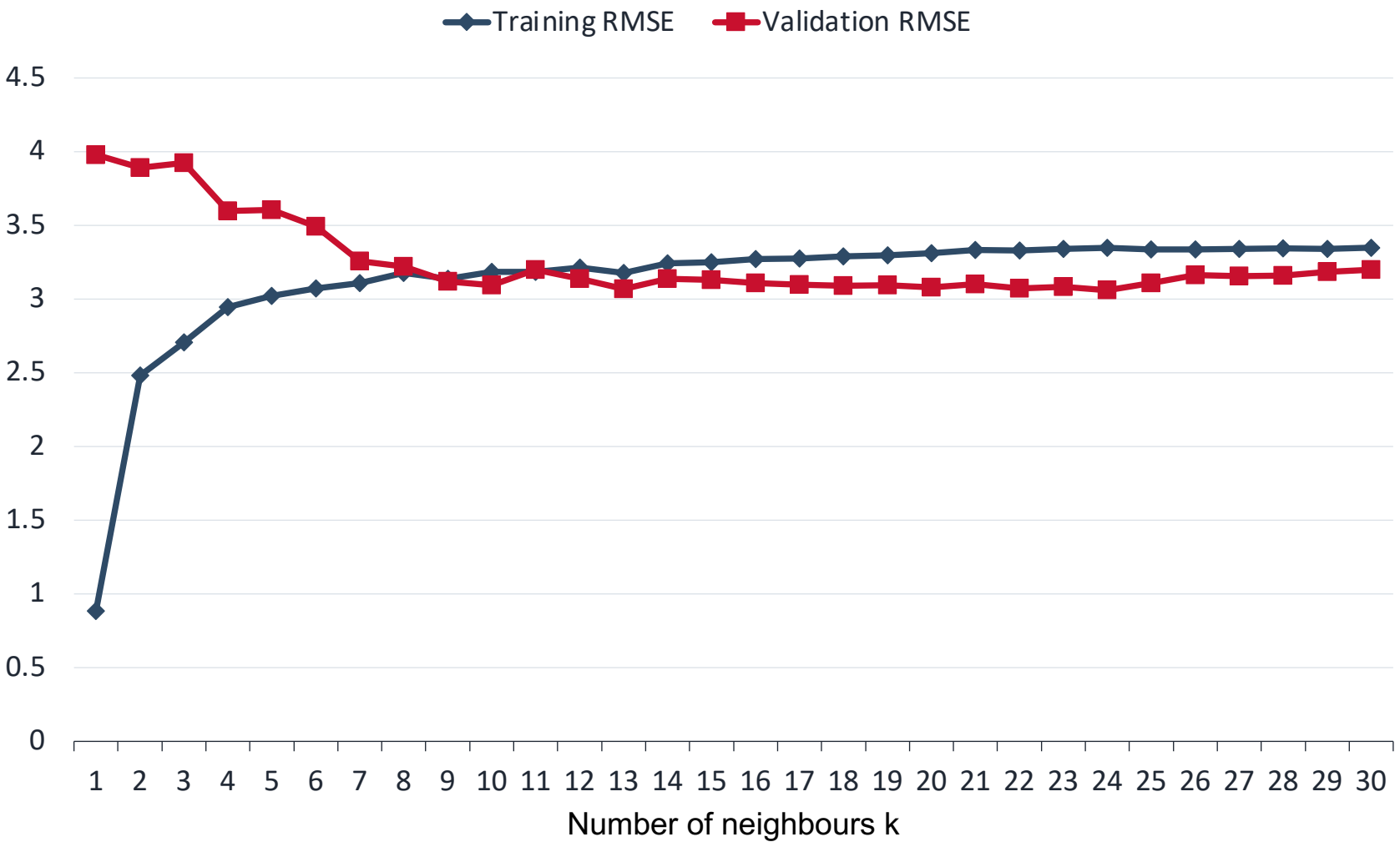
# Training, validation and final test data




Several candidates are compared on validation data. The selected validation score is optimistic because the minimum was chosen. The final test set remains unused until the specification is fixed.

- *Validation data* support model selection; **test data support final evaluation** and must not be repeatedly inspected during revision.
  - Cross-validation reuses training data for selection; independent test evaluation or nested cross-validation assesses the selected procedure.

# Selecting the number of neighbours by validation RMSE




- The TV-only model uses **150 training and 50 validation markets**.
- Training RMSE is zero at k = 1 and **generally increases with k**.
- Validation RMSE is lowest at **k = 24** (3.06).
- The training-mean baseline has RMSE 5.47.
- In units instead of thousands, every RMSE would be **1000 times larger**.
  - These markets are used for tuning; an independent test set is required for an unbiased final assessment.

# The coefficient of determination $R^2$

> $R^2$ compares the squared errors of the model with those of the mean model $\hat{y} = \bar{y}$:

$$R^2 = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}}$$

$$\mathrm{SSE} = \sum_i (y_i - \hat{y}_i)^2$$

squared errors of the model

$$\mathrm{SST} = \sum_i (y_i - \bar{y})^2$$

squared errors of the mean model

- $R^2 = 1$ is perfect; **$R^2 = 0$ matches the evaluation mean**; $R^2$ may be negative.
  - The selected kNN model has validation $R^2 = 0.67$.
  - Constant responses make $R^2$ undefined; without an intercept, training $R^2$ may be negative.
  - $R^2$ is descriptive, not evidence of a causal relationship.

# Worked example: residuals against the baseline

- The upper panel shows the residuals of the line; their squares sum to **2.4**.
- The lower panel shows the deviations from the mean; their squares sum to **6.0**.
- $R^2$ compares the two sums: 1 − 2.4 / 6.0 = **0.60**.
  - The line removes 60 % of the squared deviation that the mean leaves.
- The line $\hat{y} = 2.2 + 0.6x$ is specified for this example.
  - Section 04 shows that it is **the least-squares line** of these points.
- The two sums define $R^2$ for **any model**, linear or not.
- The baseline uses the evaluation-set mean in the $R^2$ definition; a deployable constant prediction instead uses **the training mean**.

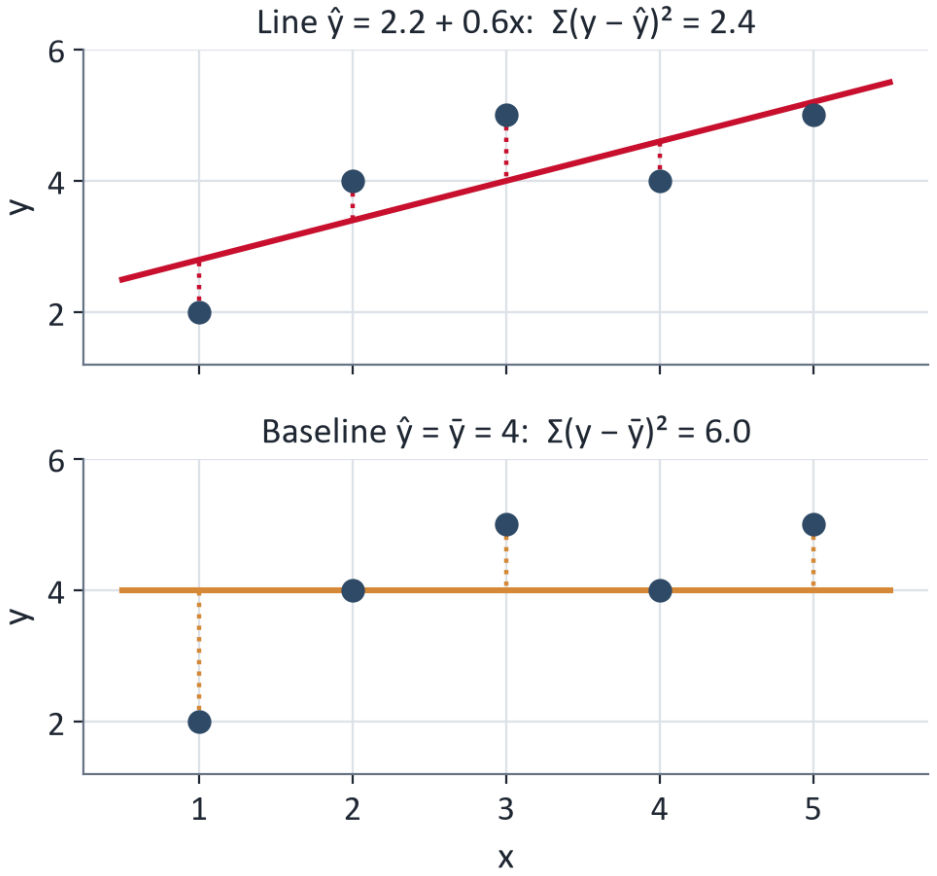



*Figure 6 — residuals (red) and deviations (gold)*

# Reading the error measures: a summary

| Measure | In words | Units | Best value | 5-point example |
|---|---|---|---|---|
| MSE | Average squared error | units of y, squared | 0 | 0.48 |
| RMSE | Typical size of an error | same as y | 0 | 0.69 |
| MAE | Average absolute error; less affected by a few big misses | same as y | 0 | 0.64 |
| $R^2$ | Share of the variation in y that the model explains, compared with predicting the mean | none | 1 | 0.60 |

**1 Compare**

Always with a baseline: the mean model predicts $\bar{y}$ for everyone and has $R^2 = 0$.

**2 Interpret**

$R^2 = 0.60$: the line removes 60 % of the squared error of the mean model.

**3 Report**

Errors on test data, never only on the training data.

# 04

# Linear regression and its estimation

A model with two coefficients, and two ways of finding the coefficients that minimise the MSE of Section 03



# The simple linear regression model

- *Simple linear regression* assumes $Y \approx \beta_0 + \beta_1 X$: a straight line with **intercept $\beta_0$ and slope $\beta_1$**.
- The *residual* of observation i is $e_i = y_i - \beta_0 - \beta_1 x_i$, the vertical distance to the line.
- *Ordinary least squares* (OLS) minimises the loss $L(\beta_0, \beta_1) = (1/n)\,\Sigma_i\, e_i^2$, **the training MSE**.
  - Of the three lines in the figure, the red one has the smallest MSE, 10.5.
- Squaring the residuals makes the loss **smooth and differentiable**, which the methods below exploit.
- Two estimation methods follow: the closed form and gradient descent.
- Fitting the line is a numerical task; **valid inference** additionally requires a sampling model.

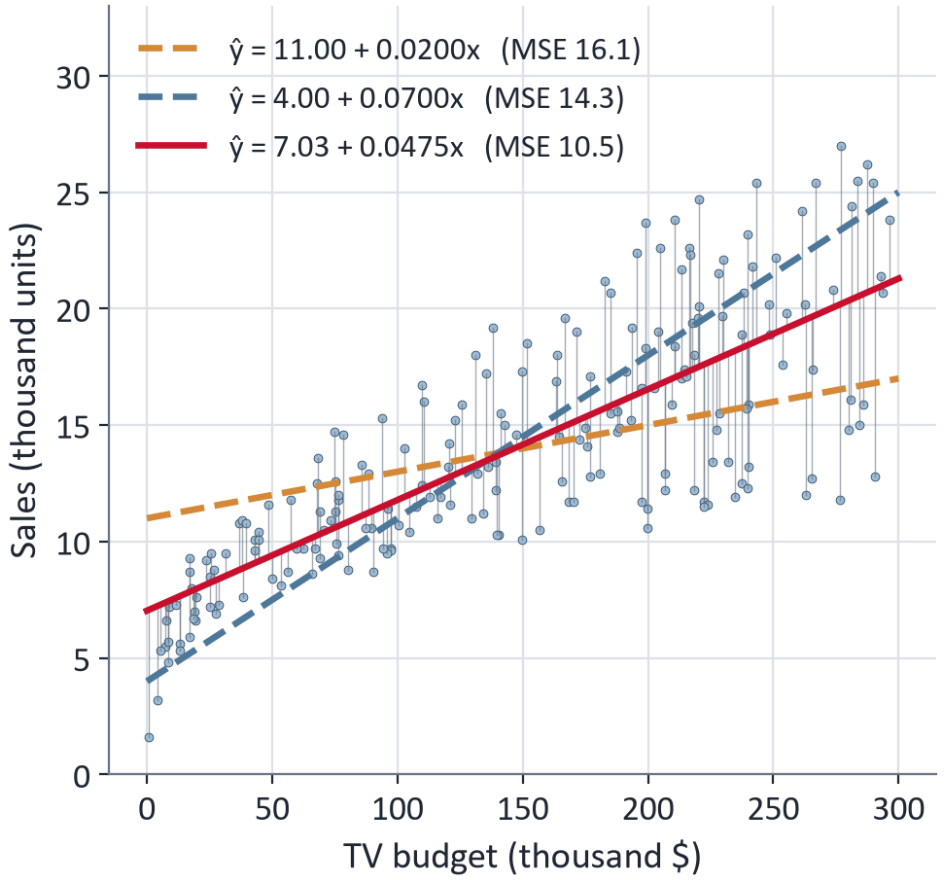



*Figure 7 — three candidate lines and their MSE*

# Reading a fitted line: intercept and slope

- The least-squares line for TV: **$\hat{y}$ = 7.03 + 0.0475 · TV**, TV in thousand $, sales in thousand units.
- *Intercept* $\beta_0$ = 7.03: the predicted sales with **zero** TV budget, about 7,033 units.
- *Slope* $\beta_1$ = 0.0475: each extra **1,000 $** of TV goes with about **47.5 more units** sold.
- Prediction: TV = 100 → $\hat{y}$ = 7.03 + 4.75 = **11.79** thousand units.
- Be careful outside the data: a line fitted on budgets 0–300 says little about a budget of 1,000.

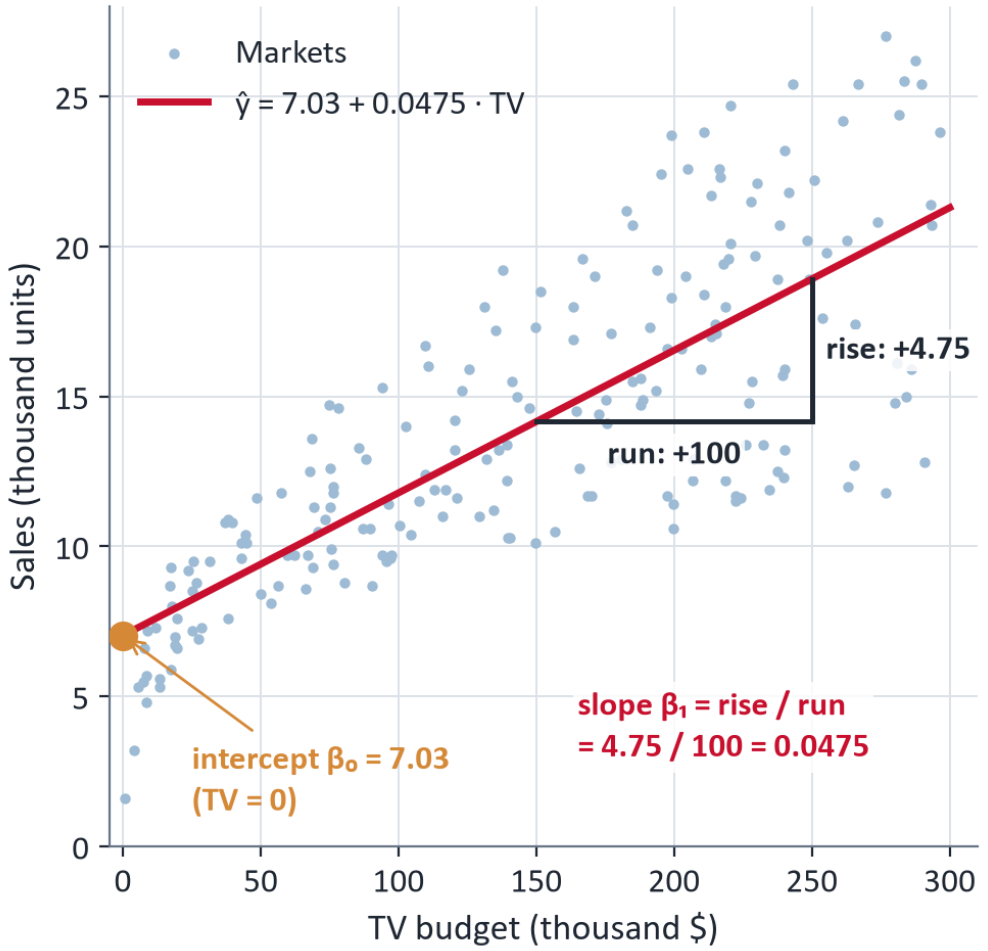



*Figure 8 — the slope as rise over run*

# The closed-form least-squares solution

> Setting both derivatives of the training MSE to zero gives:

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

slope: how x and y move together, divided by the spread of x

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

intercept: the line passes through the point ($\bar{x}$, $\bar{y}$)

- For TV, $\hat{\beta}_0 = 7.0326$ and $\hat{\beta}_1 = 0.0475$.
  - The fitted line passes through ($\bar{x}$, $\bar{y}$).
  - If x varies, convexity gives a unique global minimum; a constant x makes the slope unidentified.
  - Stable implementations solve least squares rather than invert a matrix.

# Worked example: the least-squares line by hand

| x | y | $x - \bar{x}$ | $y - \bar{y}$ | $(x - \bar{x})(y - \bar{y})$ | $(x - \bar{x})^2$ |
|---|---|---|---|---|---|
| 1 | 2 | −2 | −2 | 4 | 4 |
| 2 | 4 | −1 | 0 | 0 | 1 |
| 3 | 5 | 0 | +1 | 0 | 0 |
| 4 | 4 | +1 | 0 | 0 | 1 |
| 5 | 5 | +2 | +1 | 2 | 4 |
| **Σ = 15** | **Σ = 20** | **0** | **0** | **6** | **10** |

1. **Means**
   $\bar{x} = 15 / 5 = 3,\ \bar{y} = 20 / 5 = 4$

2. **Slope**
   $\hat{\beta}_1 = 6 / 10 = 0.6$

3. **Intercept**
   $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = 4 - 0.6 \cdot 3 = 2.2$

4. **Line**
   $\hat{y} = 2.2 + 0.6x$, with $\Sigma e^2 = 2.40$

- The products $(x - \bar{x})(y - \bar{y})$ are **positive** when x and y are on the same side of their means: the slope measures how x and y **move together**.
- No other straight line through these points has a smaller sum of squared errors than 2.40.

# Multiple linear regression and the design matrix

| y | | 1 | TV | radio | newspaper | | β | | ε |
|---|---|---|---|---|---|---|---|---|---|
| 22.1 | | 1 | 230.1 | 37.8 | 69.2 | | $\beta_0$ | | $\varepsilon_1$ |
| 10.4 | | 1 | 44.5 | 39.3 | 45.1 | | $\beta_1$ | | $\varepsilon_2$ |
| 9.3 | = | 1 | 17.2 | 45.9 | 69.3 | | $\beta_2$ | + | $\varepsilon_3$ |
| 18.5 | | 1 | 151.5 | 41.3 | 58.5 | | $\beta_3$ | | $\varepsilon_4$ |
| ⋮ | | ⋮ | ⋮ | ⋮ | ⋮ | | | | ⋮ |
| *n × 1* | | *n × (p + 1)* | | | | | *(p + 1) × 1* | | *n × 1* |

*First four of the n = 200 markets; the column of ones carries the intercept $\beta_0$.*

**Least squares in matrix form**

**Normal equations:** $X^T X \hat{\beta} = X^T y$

**Solution:** $\hat{\beta} = (X^T X)^{-1} X^T y$

The cost is about $n \cdot p^2 + p^3$ operations, which suits moderate p.
Numerical libraries solve the system by a QR or singular-value decomposition rather than by inverting $X^T X$.

**Advertising, least-squares fit:**
**$\hat{\beta}$ = (2.939, 0.046, 0.189, −0.001)**

- *Multiple linear regression* models $Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p + \varepsilon$, in matrix form **$y = X\beta + \varepsilon$** with the *design matrix* X.
  - The normal equations have a unique solution when **the columns of X are linearly independent**; with p = 1 they reduce to the formulas of the previous slides.

# Gradient descent in plain words: walking downhill in fog

1. **Feel the slope**
   under your feet: the derivative dL/dw
2. **Step downhill**
   move against the slope
3. **Choose the step size**
   the learning rate η
4. **Stop on flat ground**
   when the derivative is about 0

| Step | w | Loss $(w - 3)^2$ | Slope $2(w - 3)$ | New $w = w - 0.25 \cdot$ slope |
|---|---|---|---|---|
| 0 | 0 | 9 | −6 | 1.5 |
| 1 | 1.5 | 2.25 | −3 | 2.25 |
| 2 | 2.25 | 0.562 | −1.5 | 2.625 |
| 3 | 2.62 | 0.141 | −0.75 | 2.812 |
| 4 | 2.81 | 0.0352 | −0.375 | 2.906 |

- Example: loss $L(w) = (w - 3)^2$, start at $w = 0$, $\eta = 0.25$. Each step **halves the distance** to the minimum $w = 3$, and the steps shrink by themselves.
- With $\eta = 1$ the walker jumps $0 \to 6 \to 0 \to 6$ forever; with $\eta > 1$ it climbs out of the valley.

# Gradient descent: steps against the slope

- *Gradient descent* starts from an arbitrary point and **moves repeatedly against the slope** of the loss.
  - Where the derivative is negative the step goes right; where it is positive, left.
- The step is proportional to the derivative: $w \leftarrow w - \eta \cdot dL/dw$.
  - The *learning rate* $\eta > 0$ sets the step size.
- Steps shrink automatically **as the slope flattens near the minimum**.
- The method needs only derivatives, so it applies where **no closed form exists** (Lecture 9).
  - Too small an $\eta$ wastes iterations; too large an $\eta$ overshoots.
- For this quadratic, convergence is possible from either starting side when **the learning rate is sufficiently small**.

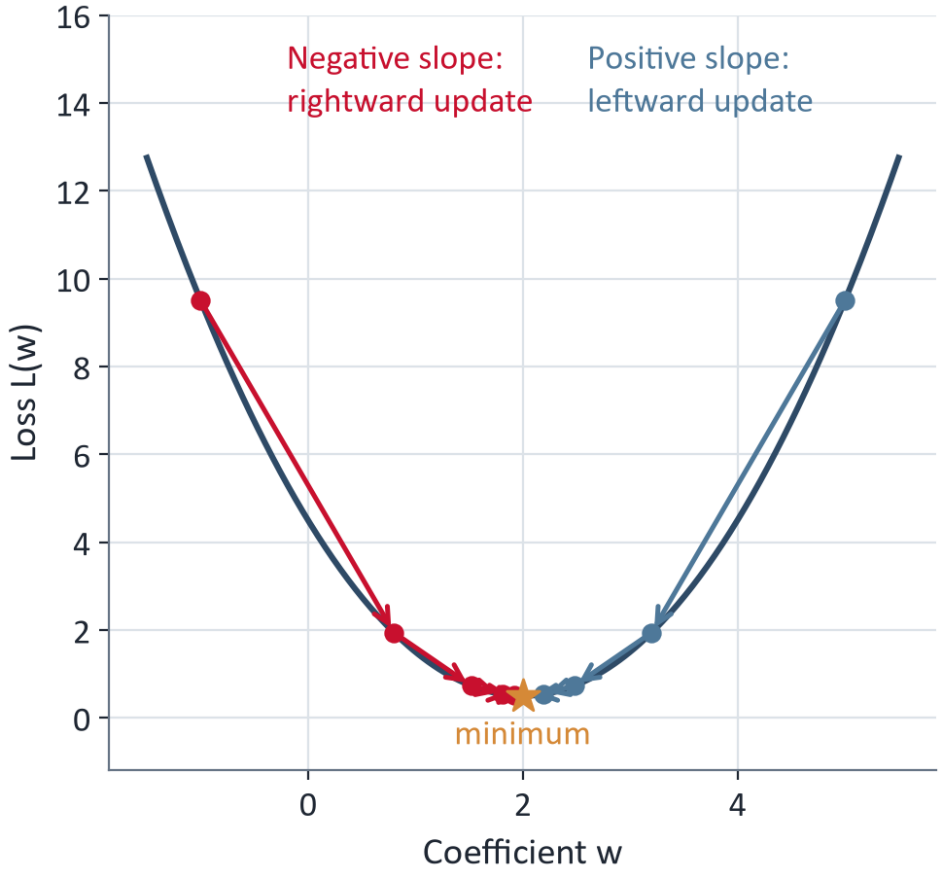



*Figure 9 — two descents with $\eta = 0.3$*

# The gradient-descent update for linear regression

For the MSE loss, the two partial derivatives are:

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n}\sum_i (y_i - \hat{y}_i) \qquad \frac{\partial L}{\partial \beta_1} = -\frac{2}{n}\sum_i x_i (y_i - \hat{y}_i)$$

- Both coefficients are **updated simultaneously** from the same residuals.
  - Each derivative is the average of the residuals, weighted by 1 or by $x_i$.
- The loop stops when the gradient is close to zero or **an iteration budget** is exhausted.
  - Each iteration costs one pass over the n observations.
- *Stochastic gradient descent* estimates the gradient from one random row; mini-batch descent uses a subset of rows (Lecture 9).

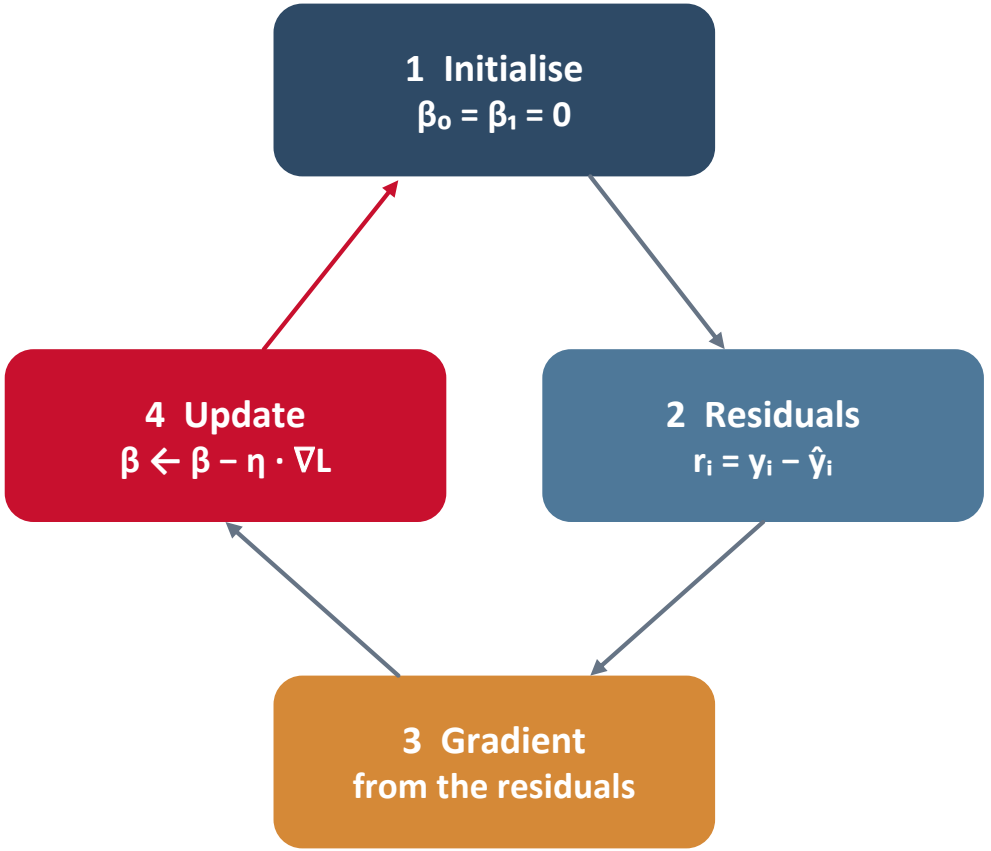
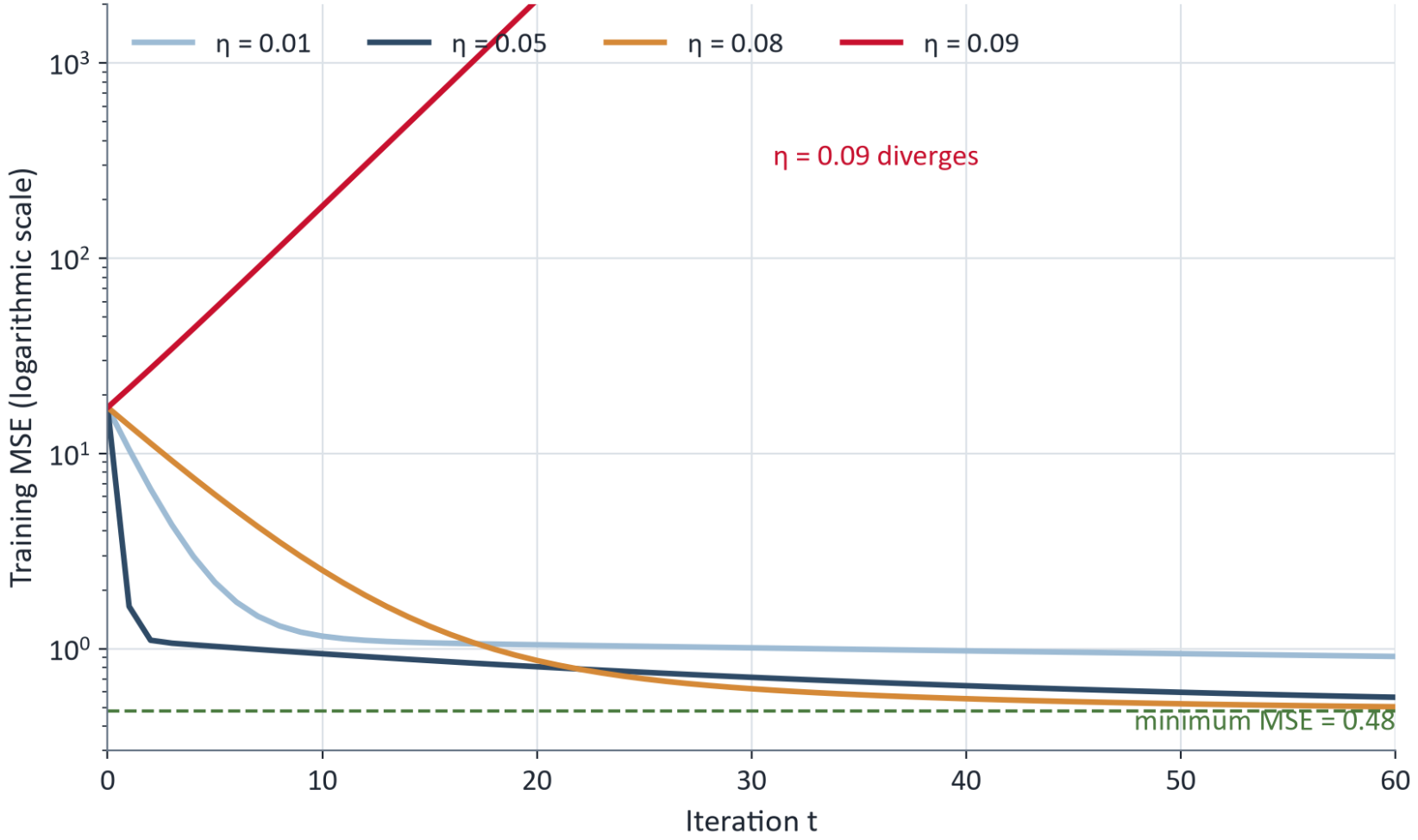



*Figure 10 — the gradient-descent loop*

# The learning rate: convergence and divergence




- The figure compares four learning rates.
- A rate of **η = 0.01** converges slowly.
- A rate of **η = 0.05** converges steadily.
- At η = 0.08, coefficients oscillate while the loss converges.
- Convergence requires **0 < η < 0.085** here; η = 0.09 diverges from (0, 0).
- The minimum itself does not depend on η.
  - The limit depends on the curvature of the loss, so predictors are standardised first.

05

# Polynomial regression and overfitting

Polynomial features extend least squares; bias and variance explain the prediction error of increasingly flexible fits.



# Polynomial regression as linear regression

- *Polynomial regression* of degree d fits $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_d\, x^d$.
- The model is **linear in the coefficients**, so least squares applies unchanged.
  - The features $x, x^2, \ldots, x^d$ are **new columns of the design matrix X**, computed from x before fitting.
- On curved data the cubic fit **follows the pattern** that a straight line misses.
  - Thirty points simulated from a sine curve plus random noise.
- Higher degrees bend more, at the cost of **more coefficients to estimate**.
  - The following examples compare approximation accuracy with sampling variability as degree increases.
- A degree-d model contains **d + 1 coefficients**, including the intercept.

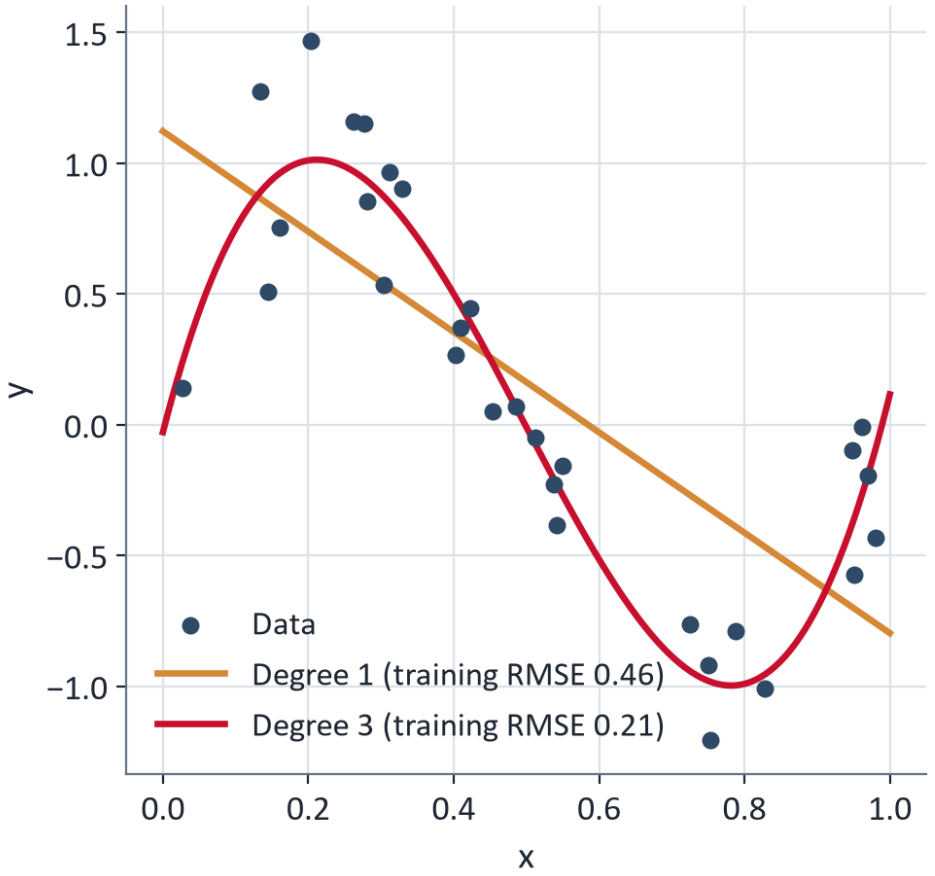
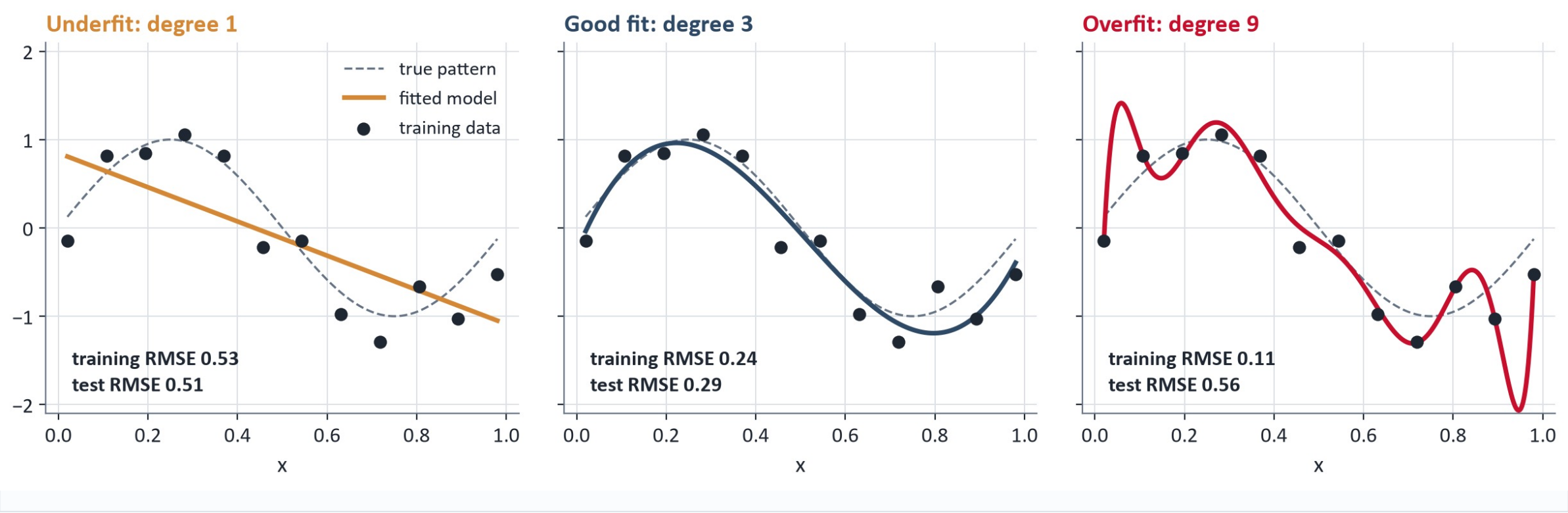


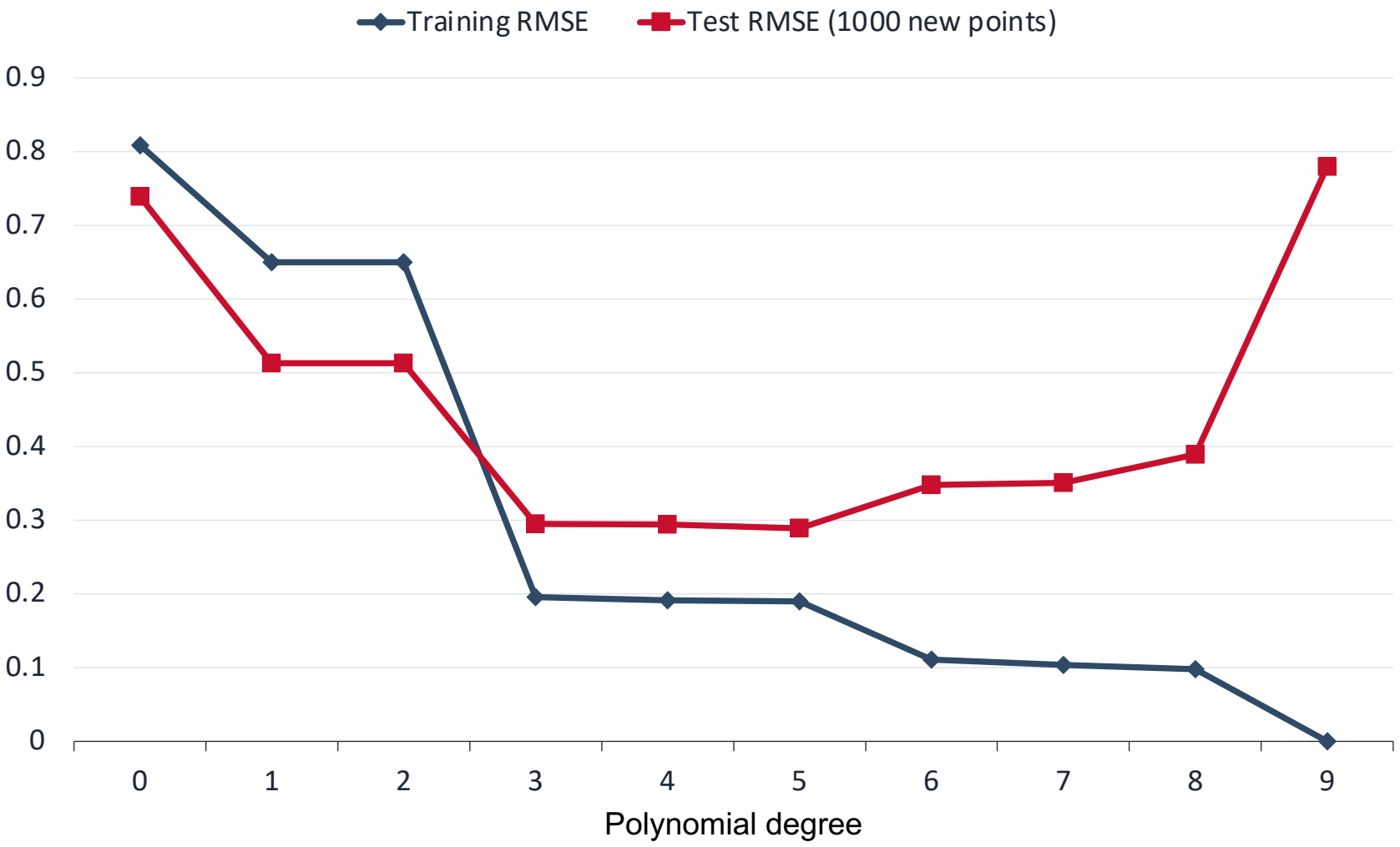

*Figure 11 — degree 1 versus degree 3*

# Underfitting, a good fit and overfitting




- **Underfit**: too simple, misses the pattern; large error on training *and* test data, like a student who did not study.
- **Good fit**: follows the pattern, ignores the noise, like a student who understood the material.
- **Overfit**: bends through every training point, noise included; tiny training error, large test error, like a student who memorised the answers.

# Polynomial degree and held-out prediction error




- For nested polynomial families, optimal training error **cannot increase** with degree and reaches numerical zero at degree 9 here.
- Held-out RMSE is smallest at **degree 5** and increases at higher degrees.
- Training error alone **cannot select the degree**.
- Selection on the held-out curve makes it a validation set; **final evaluation needs independent data**.
- The gap suggests overfitting but is not a universal measure.

# Bias and variance: the dartboard picture

- Imagine fitting the same kind of model on **many different samples**; each fit is one dart thrown at the bullseye, the true f.
- *Bias*: the darts are **systematically off-centre**; the model is too simple (degree 1).
- *Variance*: the darts are **scattered**; the model changes a lot from sample to sample (degree 9).
- More flexibility usually means **lower bias but higher variance**; the best model balances the two.
- Some error remains whatever the model: the noise $\sigma^2$ in the data.

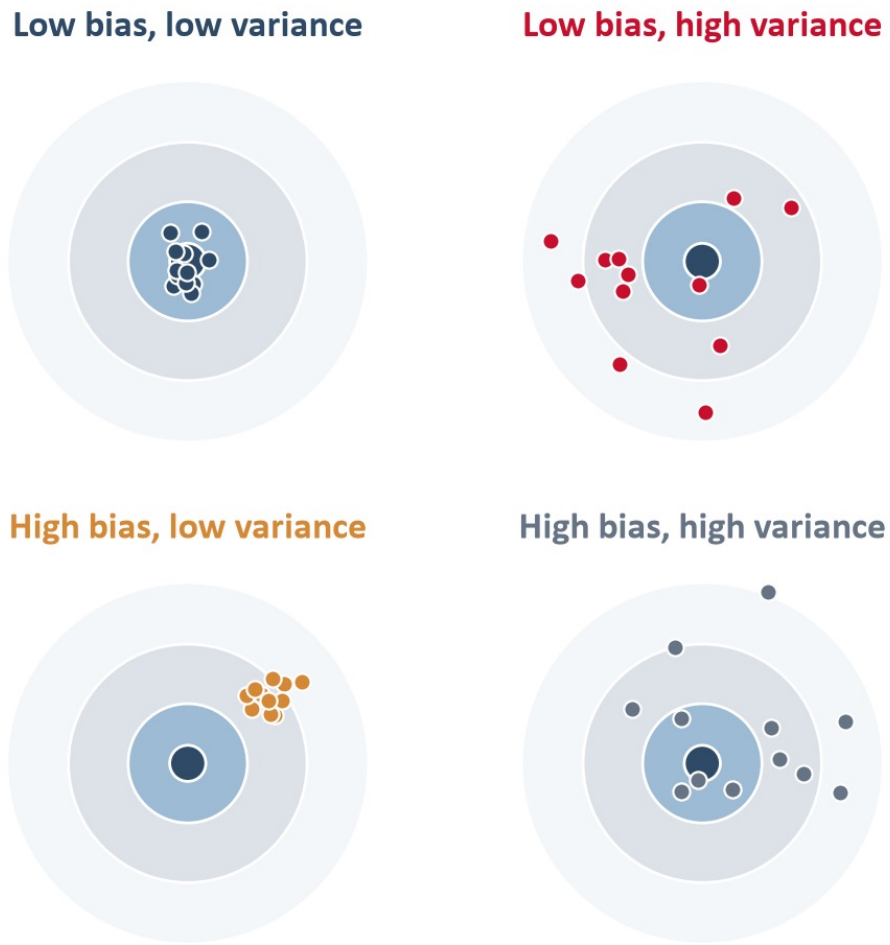



*Figure 12 — four combinations of bias and variance*

# The bias–variance trade-off

- Over repeated training sets, the expected test error at x splits into **bias² + variance + σ²**.
  - *Bias* $E[\hat{f}(x)] - f(x)$ is the systematic error of the average fit; *variance* is the spread of $\hat{f}(x)$ around that average.
  - Noise variance is irreducible for these predictors and squared-error prediction; additional informative predictors may reduce it.
- In this simulation, degree 1 has bias² = 0.22 and variance = **0.012**.
- At degree 9, variance = **0.42**, 34 times that of degree 1.
- Averaged over training sets, the sum is smallest at **degree 3** in this simulation, not at a universal degree.
  - 500 training sets of the ten x values of the previous slides, σ = 0.25.

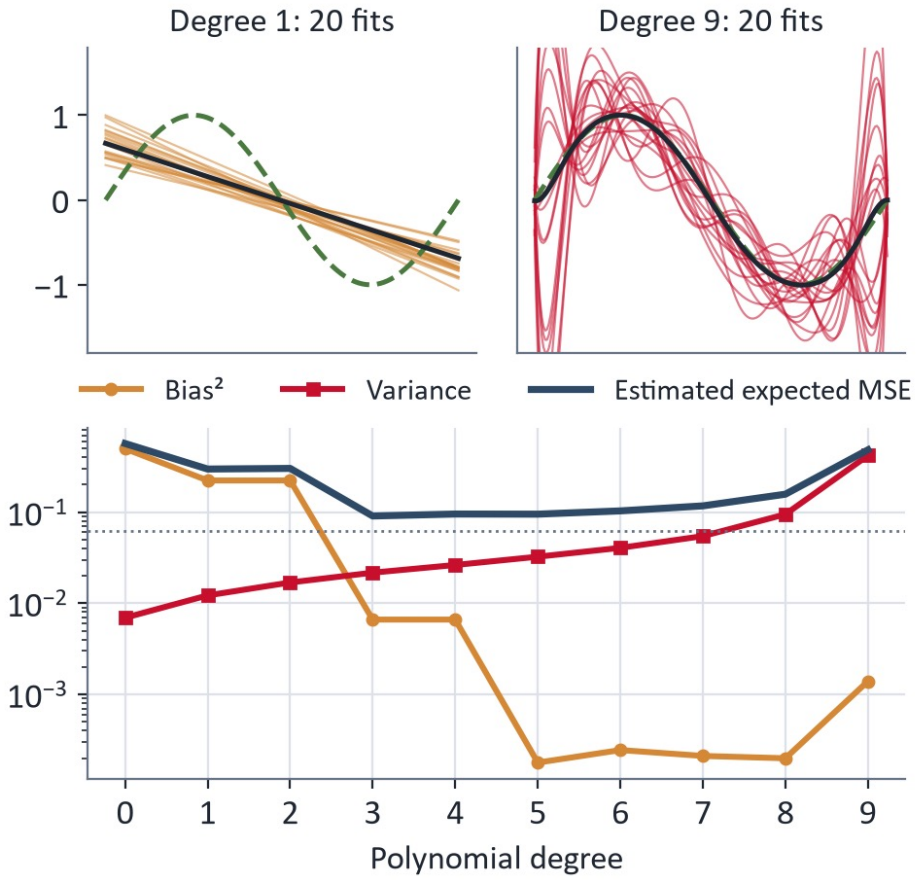



*Figure 13 — bias and variance by polynomial degree*

06

# Regularisation

Controlling the overfitting of Section 05 by penalising large coefficients, with the penalty chosen by cross-validation



# Regularisation in plain words: a price on large coefficients

**Total cost = training error + λ × size of the coefficients**

**1 λ = 0**

No price: ordinary least squares. A flexible model may overfit.

**2 λ moderate**

Coefficients shrink, the curve calms down; often the best test error.

**3 λ very large**

All slopes pushed close to 0; the model predicts almost the mean and underfits.

- Large coefficients are what make a curve wiggle; charging for them keeps the model **simple unless the data really need it**.
- λ is a hyperparameter: chosen by **cross-validation**, never on the test set. Predictors are standardised so that one price fits all.

# Regularisation: a penalty on coefficient size

$$L(\beta) = \underbrace{\tfrac{1}{2}\,\mathrm{MSE}(\beta)}_{\text{error on the training data}} + \underbrace{\lambda \cdot R(\beta)}_{\substack{\text{strength} \\ \text{penalty: size of the coefficients}}}$$

- *Regularisation* adds a **penalty** R(β) to the training error; R grows with the size of the coefficients.
  - The *regularisation parameter* λ ≥ 0 sets the trade-off; λ = 0 returns ordinary least squares.
- Predictors are **standardised first**, so that one penalty treats all coefficients alike.
  - The intercept is not penalised.
- On the degree-9 polynomial the penalty lowers the test RMSE from 0.78 to **0.36**.
  - A small penalty already removes the oscillation of the λ = 0 fit.
- Regularisation can introduce bias while **reducing variance** (Section 05).
- An excessive penalty can **underfit**; the validation objective determines whether shrinkage improves prediction.

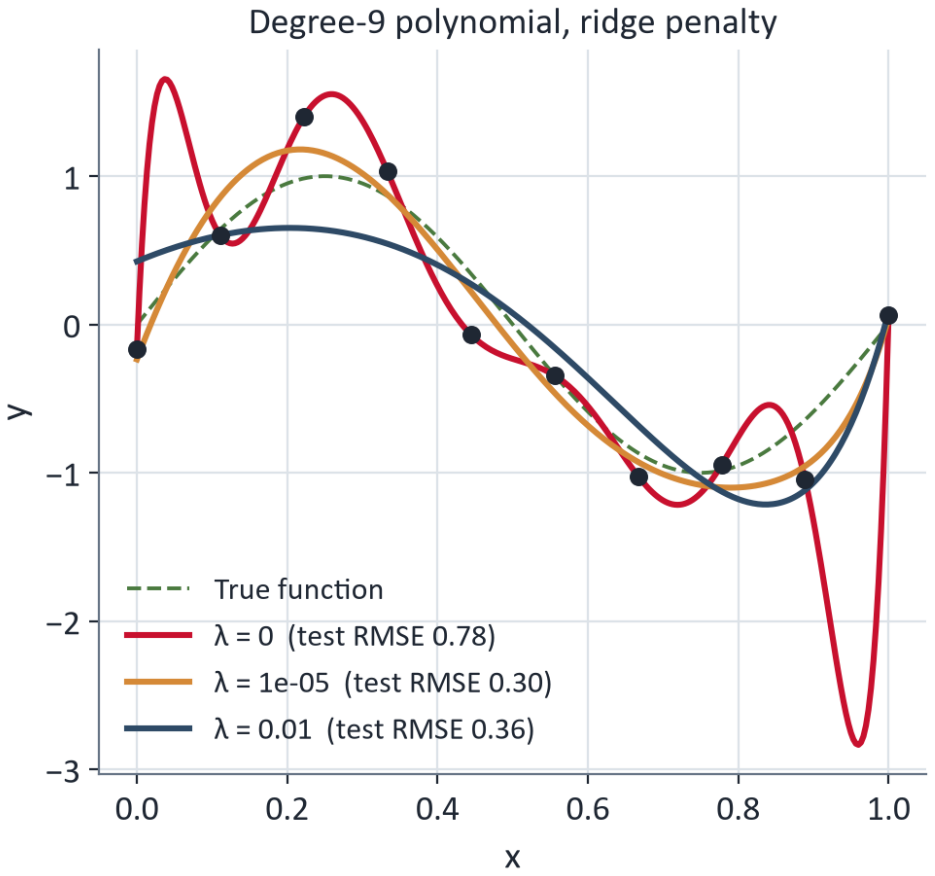



*Figure 14 — one model, three penalty strengths*

# Ridge regression and the L2 penalty

$$\text{minimise} \quad \underbrace{\frac{1}{2n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}_{\text{error on the training data}} + \underbrace{\frac{\lambda}{2}\sum_{j=1}^{p}\beta_j^2}_{\text{price on the squared slopes}}$$

- *Ridge regression* = least squares **plus a price on the squared size** of every slope.
- λ = 0 gives ordinary least squares; the **larger λ**, the more every slope is **pulled towards 0**.
- Slopes shrink **smoothly** but almost never reach exactly 0: **every predictor stays** in the model.
- Useful with **many predictors** or **strongly correlated** ones: the fitted slopes become more stable.
- Figure: diabetes data, 442 patients, 10 predictors (bmi = body mass index, bp = blood pressure, s5 = log triglycerides); from left to right λ grows and **all slopes shrink**.

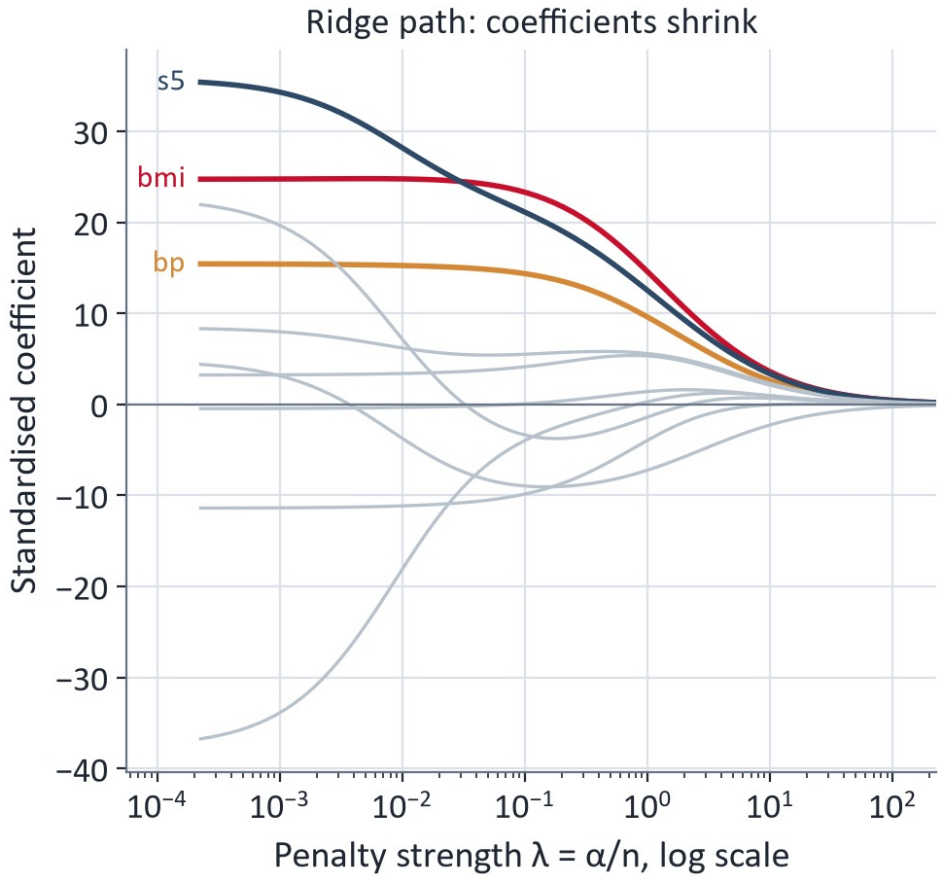



*Figure 15 — ridge coefficient path, diabetes data*

# Lasso regression and the L1 penalty

$$\text{minimise} \quad \underbrace{\frac{1}{2n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}_{\text{error on the training data}} + \underbrace{\lambda\sum_{j=1}^{p}|\beta_j|}_{\text{price on the absolute slopes}}$$

- The *lasso* = least squares **plus a price on the absolute size** |β| of every slope.
- Unlike ridge, the lasso sets some slopes **exactly to 0**: those predictors **drop out** of the model.
- So the lasso also **selects variables** and gives a shorter model that is easier to read.
- Figure: as λ grows (left to right) predictors leave one by one; **bmi, s5 and bp** stay longest, so they matter most for prediction.
- A slope of 0 means the predictor is **not needed, given the others**; it is not proof that it has no effect.

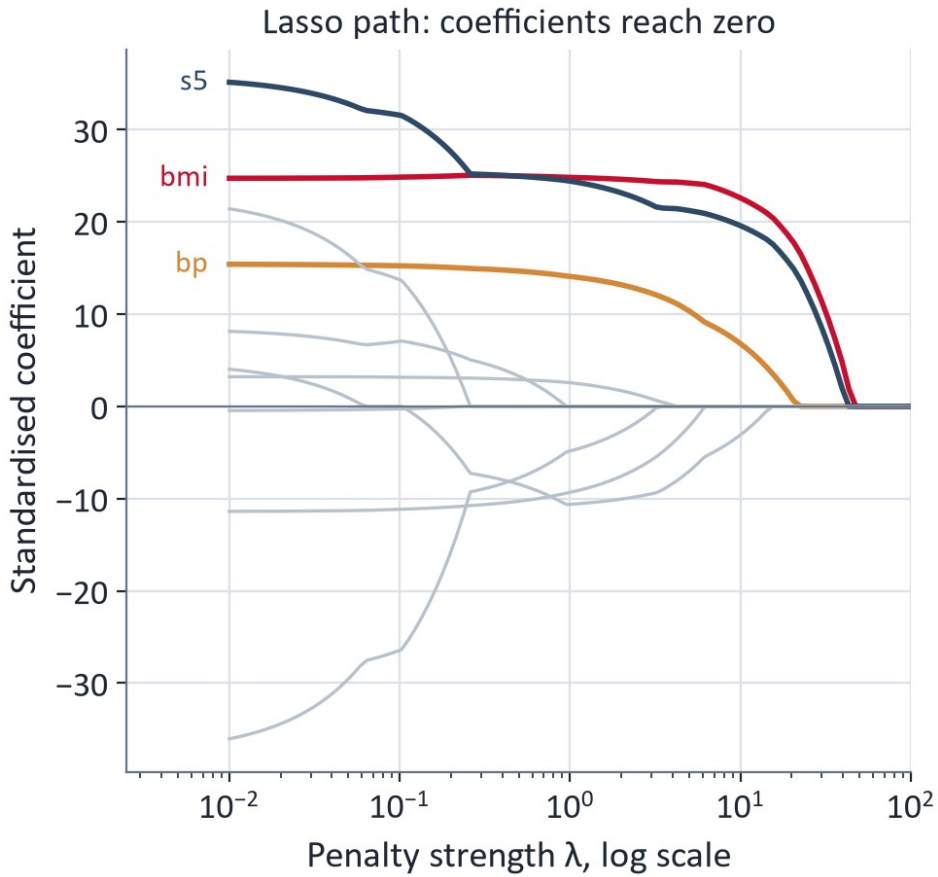



*Figure 16 — lasso coefficient path, diabetes data*

# Why ridge shrinks but the lasso can remove a predictor

- Ridge charges **$\beta^2$**: a small slope is almost free ($0.1^2 = 0.01$), a large one is expensive ($3^2 = 9$). Ridge cuts large slopes hard but has **no reason to push small ones to exactly 0**.
- The lasso charges **$|\beta|$**: every unit costs the same, even the last one (0.1 costs 0.1). A predictor that does not lower the error by at least that price gets a slope of **exactly 0**.
- Two correlated predictors with a total effect of 4: ridge prefers to **share** it, since (2, 2) costs $2^2 + 2^2 = 8$ and (4, 0) costs 16; the lasso pays 4 either way and often **keeps just one**.
- Analogy: ridge is a **tax that grows with size**; the lasso is a **fixed fee per unit**, so small, unhelpful predictors are not worth keeping.

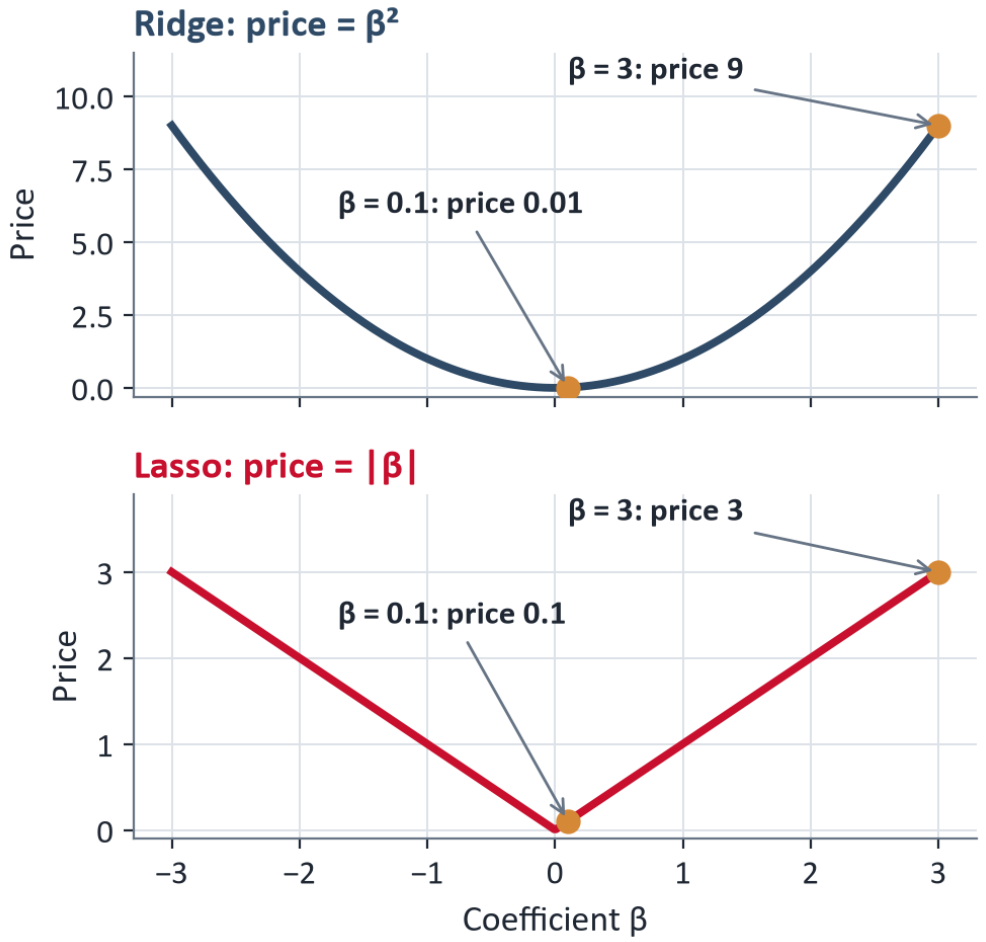



*Figure 17 — the price of one coefficient under each penalty*

# Ridge and lasso on the Advertising data

- The same 200 markets, the three budgets standardised, the same penalty **λ = 0.5** for both methods.
- *Least squares*: TV 3.92, radio 2.79, newspaper −0.02; newspaper adds almost nothing.
- *Ridge*: **every slope is pulled in** (TV 3.92 → 2.64); newspaper even grows to 0.26, because it is correlated with radio (r = 0.35) and ridge **shares** weight between them.
- *Lasso*: newspaper is set to **exactly 0** and leaves the model; TV and radio each lose about 0.47.
- Both penalties make the training error a little worse on purpose; whether they help is decided on **new data**, by cross-validation.

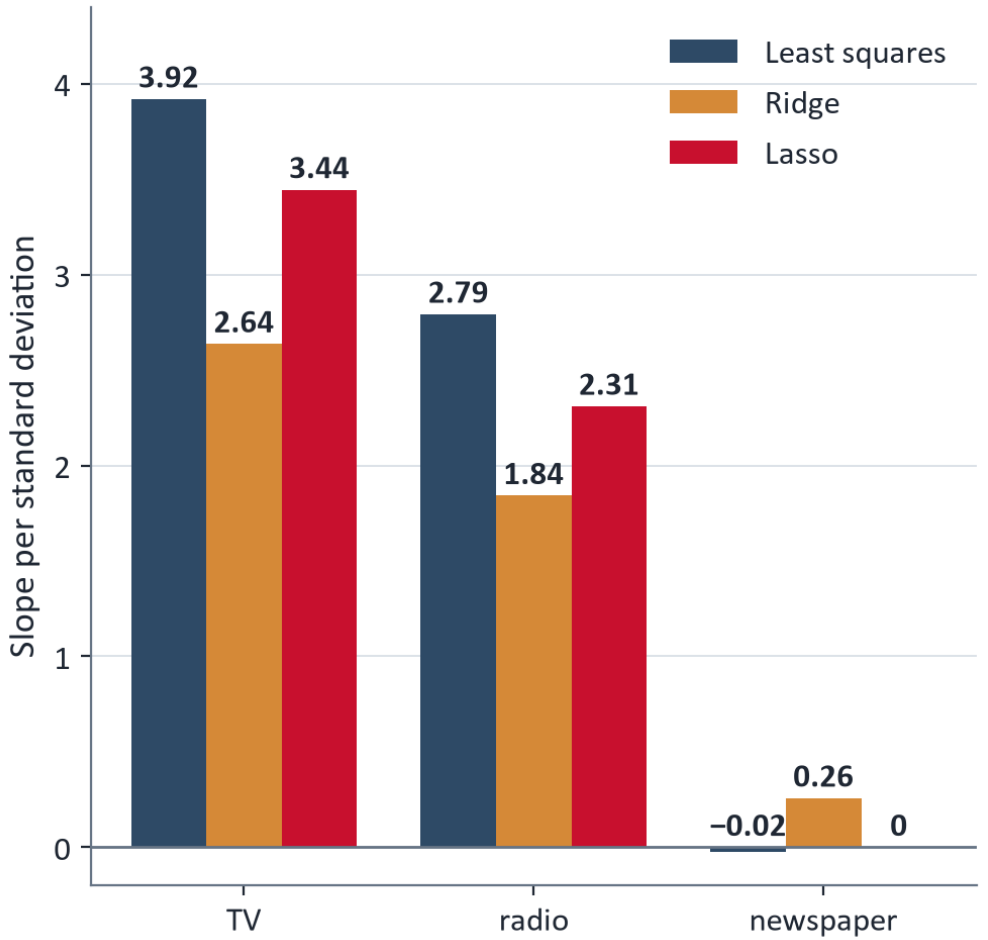


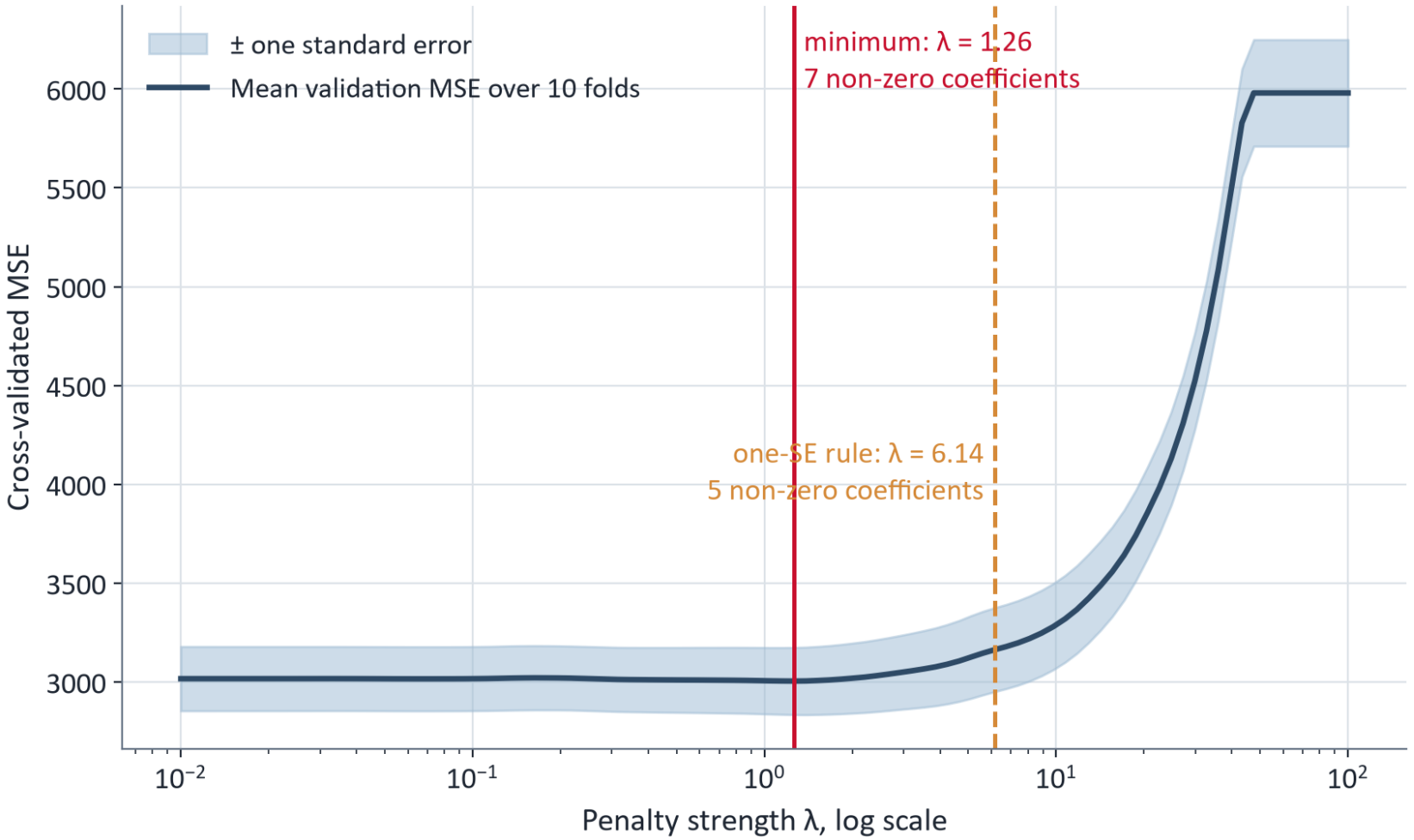

*Figure 18 — standardised slopes of the three methods, λ = 0.5*

# Ridge, lasso and elastic net side by side

| | Ridge (L2) | Lasso (L1) | Elastic net |
|---|---|---|---|
| Penalty | $\lambda \Sigma \beta_j^2$ | $\lambda \Sigma \lvert\beta_j\rvert$ | a mix of the two, weight ρ on L1 |
| Effect on coefficients | Shrinks all towards 0, rarely exactly 0 | Shrinks, and sets some **exactly to 0** | Sets some to 0, shrinks the rest |
| Selects variables? | No | Yes | Yes |
| Correlated predictors | Shares the weight among them | Tends to keep one and drop the others | Keeps the group together |
| Good when | Many predictors, each with a small effect | Only a few predictors really matter | Many correlated predictors |
| Hyperparameters | λ | λ | λ and ρ (ρ = 1 is the lasso) |

In every case: standardise the predictors, do not penalise the intercept, and choose λ by cross-validation on the training data.

# Choosing λ by cross-validation




- In *k-fold cross-validation* each of k folds is held out once and **the k validation errors are averaged** (Lecture 6).
- Lasso, 10 folds: the minimum is at **λ = 1.26**.
- The *one-standard-error rule* takes the largest λ within one SE: **5 predictors** remain.
- The band uses fold SD divided by √10; it is **a heuristic uncertainty summary**, not an independent-fold confidence interval.
- The test set is not used for choosing λ.

# Cross-validation of a penalty without data leakage




*Score = mean of the 5 validation scores*

1. Standardise with the statistics of the training folds only
2. Fit the whole path of coefficients over a grid of λ (and ρ)
3. Record the validation MSE of every (λ, ρ) on the held-out fold
4. Average over the folds, choose (λ, ρ), refit on all training rows

*Elastic net on the diabetes data, 10 folds, ρ ∈ {0.1, 0.5, 0.9, 1}*

| 1.26 | 1 | 3 of 10 |
|---|---|---|
| **Chosen λ** | **Chosen ρ** | **Zero coefficients** |

- *Data leakage* occurs when information from the held-out rows **enters the fitting**, for example through standardisation statistics.
  - Every preprocessing step is therefore refitted inside each fold; otherwise the cross-validated error is optimistic.

07

# A regression analysis in practice

One worked analysis of the Advertising data: workflow, evaluation, interpretation and residual diagnostics



# The regression workflow in seven steps

**1 Question**

What do we predict, for whom, and how is success measured?

**2 Split**

Put a test set aside before looking closely at the data.

**3 Explore**

Plot y against each predictor; check units and missing values.

**4 Prepare**

Standardise, create features; fit these steps on training rows only.

**5 Fit and tune**

Fit candidate models; choose k or λ by cross-validation.

**6 Evaluate**

Test RMSE and $R^2$, compared with the mean model.

**7 Interpret**

Coefficients are associations, not causes; report the limits.

- The next two slides follow these steps on the Advertising data: 150 training and 50 test markets, standardised budgets, least squares, and a test RMSE of 2.00 against 5.47 for the mean model.

# Worked example: multiple regression on test markets

- The 200 markets are split at random into **150 training and 50 test markets**.
  - The three budgets are standardised with training statistics; β is estimated by least squares.
- On the test markets **RMSE = 2.00**, against 5.47 for the mean model; $R^2 = 0.858$.
  - The training $R^2$ is 0.907; only the test value estimates the error on new markets.
- The standardised slopes are TV 3.73, radio 2.94 and newspaper **0.02**.
  - The newspaper slope is small on this standardised scale, conditional on the other budgets.
- Points far from the diagonal indicate **large additive-model errors**.
- This score describes **one prespecified split**, not uncertainty over all possible samples or robustness to changing markets.

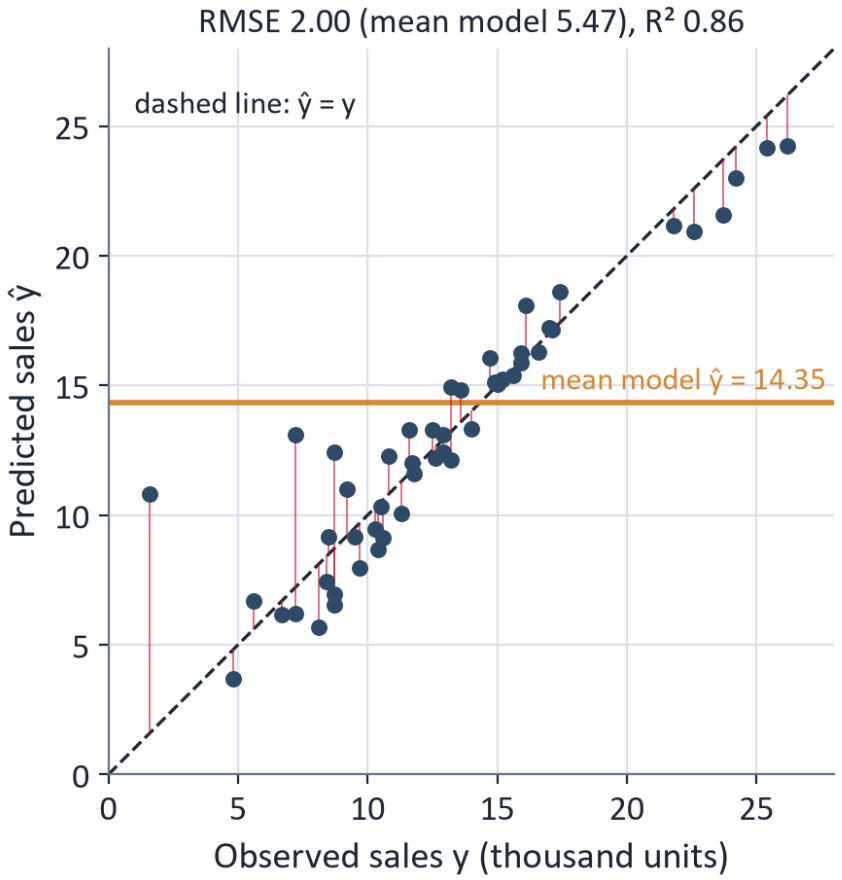
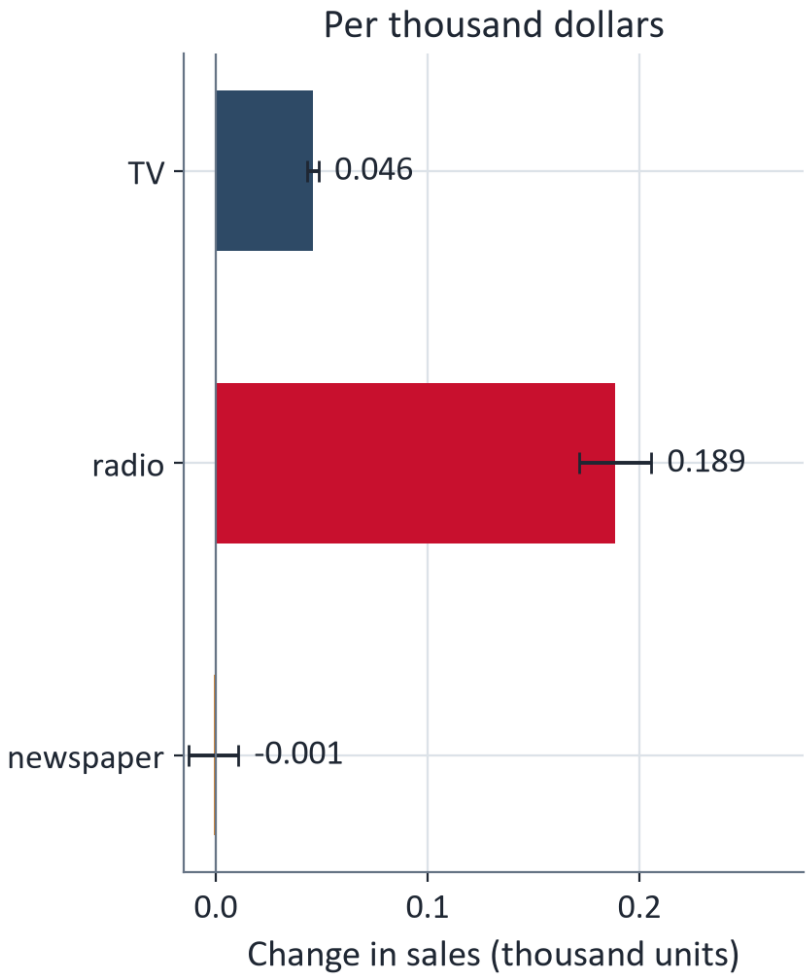
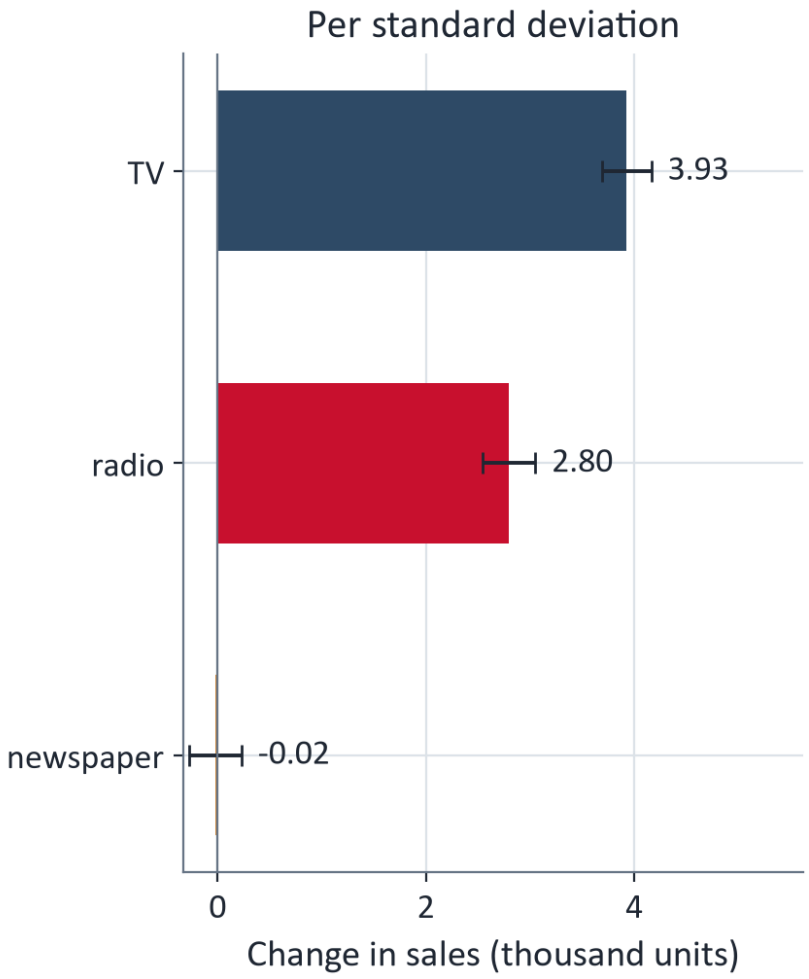



*Figure 19 — predicted against observed test sales*

# Interpreting the fitted coefficients




- One thousand dollars of radio is associated with **189 more units** sold, for fixed TV and newspaper.
- Raw coefficients depend on units, so they **cannot rank predictors**.
- Per standard deviation, the slopes are TV 3.93, radio 2.80 and newspaper -0.02.
- All slopes describe conditional associations.

# The formulas of this lecture (1/2): measuring error

| Idea | Formula | In words |
|---|---|---|
| kNN regression | $\hat{y} = \frac{1}{k} \sum_{i \in N_k} y_i$ | Average of the k most similar examples |
| Residual | $e_i = y_i - \hat{y}_i$ | How far off one prediction is |
| MSE, RMSE | $\mathrm{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2, \quad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}$ | Average squared error; RMSE is in the units of y |
| MAE | $\mathrm{MAE} = \frac{1}{n} \sum_i \lvert y_i - \hat{y}_i \rvert$ | Average absolute error |
| $R^2$ | $R^2 = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}}$ | Share of the variation explained: SSE = errors of the model, SST = errors of the mean model |

# The formulas of this lecture (2/2): fitting models

| Idea | Formula | In words |
|---|---|---|
| Linear model | $\hat{y} = \beta_0 + \beta_1 x_1 + \ \ldots \ + \beta_p x_p$ | Start value plus a slope per predictor |
| Least squares | $\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$ | The line with the smallest squared errors |
| Gradient descent | $\beta \ \leftarrow \ \beta - \eta \nabla L$ | A small step downhill, repeated |
| Ridge | $\mathrm{MSE} + \lambda \sum_j \beta_j^2$ | Error plus a price on squared slopes |
| Lasso | $\mathrm{MSE} + \lambda \sum_j \lvert\beta_j\rvert$ | Error plus a price on absolute slopes; some become 0 |

# Key takeaways

- ✓ A learning algorithm is specified by its **representation, evaluation and optimisation**.
- ✓ Prediction errors are evaluated **on held-out data**, relative to a baseline.
- ✓ Least squares has a **closed form**; gradient descent approaches the same minimum with a stable learning rate.
- ✓ More flexible models **fit the training data better** but may predict worse.
- ✓ Ridge shrinks coefficients; lasso can **set them to zero**; λ is chosen by cross-validation.
- ✓ Preprocessing is part of the model: a **pipeline** is refitted inside every cross-validation fold.
- ✓ A coefficient describes a **conditional association**, not a cause.



# Questions

Co-funded by the European Union through the EuroHPC Joint Undertaking under Grant Agreement No 101263128, and by the Ministry of Digital Transformation of the Republic of North Macedonia. Views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the EuroHPC JU.

Vezilka

EuroHPC Joint Undertaking

Co-funded by the European Union

Republic of North Macedonia
Ministry of Digital Transformation